



FLAXON

Teach a Full-Stack Project Manager

Flaxon + Teloce HTML SPA
Signals, Admin/CMS, and MinifyJS

Aldane Hutchinson

Instructor recording guide | Revision 2 | October 2026

Build protected APIs first. Connect the interface to working data. Verify the whole application before deployment.

Say, type, show: narration prompts, live-coding steps, demonstrations, editing notes, and the complete scoped-CSS source.

Contents

- 01 | Preview and setup
- 02 | Application factories and modules
- 03 | Database and migrations
- 04 | Authentication and cookie sessions
- 05 | Project APIs and ownership
- 06 | Tasks, status, dates, and filtering
- 07 | Test the backend before the frontend
- 08 | First Teloce HTML screen
- 09 | Authentication UI and typed API helper
- 10 | Components, signals, and progress
- 11 | SPA routing with data-teloce-link
- 12 | Staff Admin and model adapters
- 13 | CMS help content and publishing
- 14 | Full-stack verification and accessibility
- 15 | Production build and Render deployment
- Appendix A | Complete application source
- Appendix B | Recording your sample lesson

Read this first

This PDF is your working recording guide. Keep it beside your editor while you rehearse and record. Each chapter gives a lesson outcome, preparation, narration prompts, code to type, demonstrations, editing notes, commands, expected results, common errors, and an exercise. Read the Say prompts aloud during rehearsal, then use your own wording on camera.

The guide follows the actual course repository. Code blocks marked with a file path are copied from the application, not invented framework APIs. Study the small excerpt in a chapter, then open that file or use the complete source appendix. Type the important behavior while recording. Teach a few CSS rules in each component and provide the remainder of that component's scoped block. The completed app uses no external app.css stylesheet.

Repository: <https://github.com/aldanedev-create/FreecodeCamp-flaxon-project-manager-course>

Use main for the completed application, starter-scoped-css for the prepared welcome starter, chapter-04-auth for the authentication backend, and backend-ready for the complete API checkpoint. See course/checkpoints.md for the exact checkpoint scope and tag publication instructions. The source appendix belongs to the completed app; do not paste every final file into the first lesson at once.

This is an independent teaching project. Publication of a video by freeCodeCamp is not guaranteed. Flaxon's course-only security build is described in vendor/README.md. Real HTTPS/Render deployment, mail, media scanning, and multi-process operation were not verified in this environment.

Prepare your recording workspace

- Open a separate teaching checkout and the completed reference. Do not delete working production code to stage a lesson.
- Show the finished feature at the beginning of each chapter, then return to the lesson's starting state. Four recovery checkpoints exist; not every chapter has a separate snapshot.
- Rehearse each take once. When adapting the generated starter, use the pinned dependency files and vendor wheels from the completed repository.
- Record a short microphone test, enlarge code, close personal tabs, and hide credentials. Capture 1080p if your equipment supports it.
- Keep the editor, terminal, browser, and this PDF ready. Pause while switching views so edits are easy to follow.
- If you make a typing mistake, state the problem and correction. Cut long waits, but show the command and verified result.
- Record the 12-15-minute sample lesson first, ask learners for specific feedback, then revise the full-course takes.

Learning route

- Run and understand the starter.
- Build persistent, protected APIs and test them.
- Compile HTML components and connect them to those APIs.
- Add task reactivity and SPA navigation.

- Configure staff Admin and published help content.
- Verify the complete workflow and prepare deployment.

01 | Preview and setup

Lesson outcome

Run the finished application once, then generate the welcome starter in a separate folder.

Recording preparation

Target edited length: 15-20 minutes. This is a planning range, not a recording already made.

Before the take: Keep the finished app ready for the opening preview, an empty folder for CLI generation, and starter-scoped-css as the prepared reference. Install the pinned course wheels in the activated environment before generating.

Take 1 | Say, type, show

Say: By the end, a signed-in user can create projects, manage tasks, and see progress. Staff have a separate Admin, and help content comes from the CMS. We will build the backend first, then connect the browser to APIs we have already tested.

Type or explain on screen: The README clone, virtual-environment and dependency commands. Explain which activation command belongs to Windows and which to macOS/Linux.

Show and verify: Preview registration, one project, a completed task, and the Help page. Show staff Admin briefly without exposing passwords.

Take 2 | Say, type, show

Say: Flaxon owns the Python server. Teloce compiles our HTML components into JavaScript. MinifyJS optimizes the compiled JavaScript for production; we do not need a Node build for this project.

Type or explain on screen: In a separate folder: `flaxon new project-manager --no-venv`, then `cd project-manager` and `python management.py migrate`.

Show and verify: Run `flaxon welcome` and `flaxon welcome-status`. Open the generated welcome page and show its module folder.

Take 3 | Say, type, show

Say: The starter is our beginning, and the completed repository is our reference. The course uses pinned dependencies, including an explicitly labelled Flaxon build with CMS fixes.

Type or explain on screen: After CLI generation, remove the `app.css` stylesheet reference in `app.py`; move `Welcome` link/button rules into its existing style scoped block and add the `shell typography/reset` block. The prepared `starter-scoped-css` branch already contains these edits and pinned dependencies.

Show and verify: Point viewers to the checkpoint guide and briefly show the welcome component's existing style scoped block.

Editing and chapter handoff

Editing note: Cut package download waits, not the commands or their successful completion. Record a clean ten-second microphone test before this take.

End the chapter: State that the welcome project runs, then introduce modules as the next step.

How it works

The finished app has customer accounts, owned projects, tasks, progress, a staff Admin, and published help content. Flaxon handles HTTP, validation, authorization, persistence, and server composition. Teloce compiles HTML components into browser JavaScript; MinifyJS optimizes that JavaScript for production. Admin remains a separate server-rendered interface.

Use Python 3.12, Git, and a terminal. Basic Python, HTML, CSS, and JavaScript are prerequisites. This book introduces the small amount of TypeScript used here. The dependencies include a labelled course-only Flaxon build containing CMS fixes; it is not an official PyPI release. Install the supplied dependency files, not an unrelated latest version.

Exact code reference for this take

File: requirements-dev.txt

```
-r requirements.txt
pytest==9.1.1
httpx==0.28.1
playwright==1.51.0
```

Commands and demonstration

```
git clone https://github.com/aldanedev-create/FreecodeCamp-flaxon-project-manager-course.git
cd FreecodeCamp-flaxon-project-manager-course
python -m venv .venv
# Windows PowerShell: .\.venv\Scripts\Activate.ps1
# macOS/Linux: source .venv/bin/activate
python scripts/verify_vendor.py
python -m pip install -r requirements-dev.txt
python management.py migrate
python -m flaxon run app:app --reload
```

Expected result

Open <http://127.0.0.1:8000/login>, register, and create a project. To reproduce the original generation lesson, stop the server, move to a new parent folder, activate the installed environment, and run `flaxon new project-manager --no-venv`. Enter that generated directory, run `python management.py migrate`, then `flaxon welcome`, `flaxon welcome-status`, and `python -m flaxon run app:app --reload`. The raw generator still includes `app.css`; adapt it as shown in Take 3, or recover from the `starter-scoped-css` branch. Its welcome screen and module commands are the starter, not the completed task manager.

Common errors

A missing `flaxon` command usually means the virtual environment is inactive. Use `python -m flaxon`. If PowerShell blocks activation, run the environment Python directly (`.\venv\Scripts\python.exe`). A missing wheel usually means you ran installation outside the repository root. Do not generate over the completed repository.

Viewer exercise and wrap-up

Generate the starter under a new directory name. Find the welcome module and its custom command before changing any code.

Ask the viewer to pause, try the exercise, and compare the result with the chapter's expected behavior.

02 | Application factories and modules

Lesson outcome

Understand how one application composes independent features.

Recording preparation

Target edited length: 15-20 minutes. This is a planning range, not a recording already made.

Before the take: Open the generated starter and keep the final app.py available in a separate reference window. Do not mount a feature before its imports and module exist.

Take 1 | Say, type, show

Say: A feature can keep its Python routes, browser screens, and commands together. The application factory decides which features are mounted and where their APIs begin.

Type or explain on screen: create_app() and the welcome module mount in app.py. Introduce settings.py and the module directory structure.

Show and verify: Trace GET /api/welcome/status from the application mount to the module decorator.

Take 2 | Say, type, show

Say: A prefix is added once. A slash inside a project module will become /api/projects/ when we mount that module. Tests can create a fresh app against a temporary database.

Type or explain on screen: The FlaxonModule declaration in modules/projects/module.py as a preview; save its UI mapping for the frontend chapter.

Show and verify: Contrast a Python route parameter with a browser :id pattern. Show the module-owned files rather than inventing a new routing API.

Editing and chapter handoff

Editing note: Keep the architecture explanation short. Zoom in on one mount and one route; do not tour every framework file.

End the chapter: Ask the learner to find the welcome API and its module command.

How it works

A module collects routes and, when needed, its interface files. An API prefix is mounted by the application factory. The project module can own both `/api/projects/` and its browser pages without mixing browser authorization with server authorization. `create_app()` allows tests to use disposable databases. Configuration comes from settings and environment variables; production startup rejects a missing secret.

Start with the welcome module, then add auth, projects, tasks, and content as the chapters introduce them. The complete factory in the source appendix is the final composition, so it naturally contains features taught later.

Exact code reference for this take

File: `modules/projects/module.py`

```
projects = FlaxonModule(
    "projects",
    ui_dir=Path(__file__).parent / "ui",
    ui_routes={
        "ProjectList.js": "/projects",
        "ProjectDetails/[id].js": "/projects/:id",
    },
)
```

Commands and demonstration

```
python -m flaxon run app:app --reload
# In a second terminal:
curl http://127.0.0.1:8000/api/welcome/status
```

Expected result

The welcome route responds and module routes receive their mounted prefixes. The completed project API is protected. Teloce UI route patterns use `:id`; Python routes use `<int:project_id>`.

Common errors

Do not repeat `/api/projects` inside module decorators: the mount adds it. A wrong relative import can stop startup. Keep feature-specific pages in their module UI folder and shared helpers in root `ui/`.

Viewer exercise and wrap-up

Explain which file you would edit to change a project route, its screen, and its API prefix.

Ask the viewer to pause, try the exercise, and compare the result with the chapter's expected behavior.

03 | Database and migrations

Lesson outcome

Persist users, projects, tasks, sessions, and authentication attempts.

Recording preparation

Target edited length: 20-25 minutes. This is a planning range, not a recording already made.

Before the take: Use a disposable development data directory. Open migrations/0001_initial.json, database.py, and management.py.

Take 1 | Say, type, show

Say: A user owns projects, and each project owns tasks. We store those relationships in the database, so they survive a server restart.

Type or explain on screen: The users, projects, and tasks definitions in the migration; then the session and auth-attempt tables. Explain foreign keys, cascades, and indexes.

Show and verify: Run migrate and migrate --status. Run migrate again and show that no duplicate migration is applied.

Take 2 | Say, type, show

Say: SQLite calls are synchronous. Our small repository runs them in a worker thread and opens a connection for each operation. Values use placeholders rather than being joined into SQL.

Type or explain on screen: Database.connection(), all(), one(), and execute() in that order.

Show and verify: Point to PRAGMA foreign_keys and the transaction context. Show that the database path comes from DATA_DIR, not a hardcoded cloud path.

Editing and chapter handoff

Editing note: Print or enlarge the migration SQL when explaining relationships; the JSON string is difficult to read at normal editor size.

End the chapter: Recap persistence, parameterized values, and why migrations precede API writes.

How it works

The migration establishes relationships before endpoints write records. Each project belongs to a user; each task belongs to a project. Foreign keys cascade task deletion when a project is removed. Indexes support common ownership and project queries.

The Database helper opens a connection for each operation, enables foreign keys, and executes blocking SQLite work in `asyncio.to_thread`. A connection context commits a successful operation and rolls it back on failure. SQL placeholders separate values from SQL structure. The default database files live under `data/`; deployment moves them to a persistent directory. Never delete a real database just to rerun a migration.

Exact code reference for this take

File: database.py

```
"""Small SQLite repository: parameterized SQL, explicit transactions, no global connection."""

import asyncio
import sqlite3
from contextlib import contextmanager
from pathlib import Path

class Database:
    def __init__(self, path):
        self.path = Path(path)
        self.path.parent.mkdir(parents=True, exist_ok=True)

    @contextmanager
    def connection(self):
        connection = sqlite3.connect(self.path, timeout=10)
        connection.row_factory = sqlite3.Row
        connection.execute("PRAGMA foreign_keys = ON")
        try:
            with connection:
                yield connection
        finally:
            connection.close()

    async def all(self, sql, parameters=()):
        def read():
            with self.connection() as connection:
                return [
                    dict(row) for row in connection.execute(sql, parameters).fetchall()
                ]

        return await asyncio.to_thread(read)

    async def one(self, sql, parameters=()):

        rows = await self.all(sql, parameters)
        return rows[0] if rows else None

    async def execute(self, sql, parameters=()):
        def run():
            with self.connection() as connection:
                cursor = connection.execute(sql, parameters)
                return cursor.lastrowid

        return await asyncio.to_thread(run)
```

Commands and demonstration

```
python management.py migrate
python management.py migrate --status
```

Expected result

The first run applies the initial migration; a second run applies zero new migrations. Status reports applied and pending counts. Restarting the application preserves records. The full migration JSON is included in the source appendix.

Common errors

no such table means migrations have not run against the database selected by DATA_DIR. Foreign keys must be enabled on each connection. Do not concatenate a project name into SQL. Migration SQL is application code; request values are parameters.

Viewer exercise and wrap-up

Use a temporary database to verify that deleting a project removes its tasks. Write down which foreign key implements that behavior.

Ask the viewer to pause, try the exercise, and compare the result with the chapter's expected behavior.

04 | Authentication and cookie sessions

Lesson outcome

Register, sign in, sign out, and protect API requests before building forms.

Recording preparation

Target edited length: 30-40 minutes. This is a planning range, not a recording already made.

Before the take: The schema and Database helper must run. Use chapter-04-auth as a recovery point; customer UI and staff Admin are deliberately not enabled in that checkpoint.

Take 1 | Say, type, show

Say: We never store a plain password. Argon2 hashes the password, while a random opaque session token identifies an authenticated session. Only a digest of that token is stored.

Type or explain on screen: `security.py: digest(), read_session(), require_user(), and session_response();` then the auth module register and login handlers.

Show and verify: Show GET `/api/auth/session` before login, the registration response, and the public user fields without printing password hashes.

Take 2 | Say, type, show

Say: Cookies alone are not enough for these mutations. A session-bound CSRF token is required, and login rotates both the session and that token.

Type or explain on screen: `check_csrf()`, then the logout handler. Explain `HttpOnly`, `SameSite`, `production Secure`, and `expiry` while pointing to the actual code.

Show and verify: In the HTTP client, deliberately use the old CSRF token after registration and show rejection; then use the new token successfully.

Take 3 | Say, type, show

Say: Logging out must revoke the old server session. A cookie copied before logout should stop working too.

Type or explain on screen: The logout/revocation test in the auth checkpoint, or explain the completed login/logout test.

Show and verify: Run authentication tests and show a wrong-password response. Never use a staff credential in the public registration example.

Editing and chapter handoff

Editing note: Pause after session rotation. This is a concept learners need to reproduce, not a place to speed up typing.

End the chapter: Ask viewers to replay an old cookie after logout and predict the status.

How it works

Passwords are hashed with Argon2. The browser receives a random session cookie; the database stores a digest of its token. Server-side expiry is eight hours. Registration, login, and logout revoke the previous session and issue a new session and CSRF token.

First request `/api/auth/session` to establish an anonymous session. JSON mutations require `X-CSRF-Token`; browser requests also undergo an Origin check when that header is supplied.

Authentication answers who the caller is. Ownership checks later answer which records that person may use. Public registration never creates staff Admin access.

Exact code reference for this take

File: `security.py`

```
async def require_user(request):
    session = await read_session(request)
    if not session or session["user_id"] is None:
        raise Unauthorized("Please sign in.")
    user = await request.app.course_db.one(
        "SELECT id, name, email FROM users WHERE id = ?", (session["user_id"],)
    )
    if user is None:
        raise Unauthorized("Please sign in.")
    return user

async def check_csrf(request):
    origin = request.headers.get("origin")
    if origin and origin != request.app.public_origin:
        raise Forbidden("This request came from an untrusted origin.")
    if "application/json" not in request.headers.get("content-type", "").lower():
        raise Forbidden("Send JSON with a CSRF token.")
    session = await read_session(request)
    supplied = request.headers.get("x-csrf-token", "")
    if (
        not session
        or not supplied
        or not hmac.compare_digest(session["csrf"], supplied)
    ):
        raise Forbidden("Your session expired. Reload the page and try again.")
```

Commands and demonstration

```
python -m pytest -q tests/test_api.py -k "registration or login_logout or wrong_password or rate_limit"
curl -c cookies.txt http://127.0.0.1:8000/api/auth/session
```

Expected result

The session response is `{"data":{"user":null,"csrf":"..."}}` before sign-in. Follow `docs/backend.md` or `scripts/course_api_demo.py` to register without guessing cookie behavior. After registration, use the new cookie and new token. `chapter-04-auth` is a runnable backend-only checkpoint.

Common errors

A 403 usually indicates a stale/missing token, wrong JSON Content-Type, or Origin mismatch. Use exactly `http://127.0.0.1:8000` locally if that is `PUBLIC_ORIGIN`. A 401 means the endpoint requires a signed-in user. Passwords must be 12-128 characters. Do not commit cookie jars.

Viewer exercise and wrap-up

Sign out, then replay a request with the old session. Explain why server-side revocation matters even if the browser has deleted its cookie.

Ask the viewer to pause, try the exercise, and compare the result with the chapter's expected behavior.

05 | Project APIs and ownership

Lesson outcome

Create, list, read, update, and delete only the signed-in user's projects.

Recording preparation

Target edited length: 20-25 minutes. This is a planning range, not a recording already made.

Before the take: Start from the authenticated backend. Open the project module, the HTTP demonstration, and the ownership test. Use two separate learner accounts.

Take 1 | Say, type, show

Say: The owner comes from the signed-in user. It is not a field the browser gets to choose.

Type or explain on screen: `owned_project()`, `list_projects()`, and `create_project()`, including imports, validation, SQL placeholders, and the 201 response.

Show and verify: Create a project through HTTP, list it, and show the consistent data envelope.

Take 2 | Say, type, show

Say: Authentication tells us who you are. Ownership tells us which record you may use. A private record returns 404 to someone else.

Type or explain on screen: `get_project()`, `update_project()`, and `delete_project()`. Explain the CSRF and ownership checks before mutations.

Show and verify: Run `scripts/course_api_demo.py`, then the ownership test. Show a blank name returning 400 and another account getting 404.

Editing and chapter handoff

Editing note: This is the backend half of the pilot. Keep one successful request and one denied request visible long enough to read.

End the chapter: Transition to tasks: tasks will inherit permission through their parent project.

How it works

A project name is required and limited to 120 characters; an optional description is limited to 1000. Server validation trims text. Creating a project records the owner from the authenticated session rather than a browser-supplied owner ID.

`owned_project()` combines the record ID with the current user ID. Missing and other users' projects both return 404. This avoids revealing whether a private record exists. GET lists only owned rows. PUT and DELETE check CSRF and ownership. Success responses consistently place records under `data`; creation returns HTTP 201.

Exact code reference for this take

File: `modules/projects/module.py`

```

async def owned_project(request, project_id):
    user = await require_user(request)
    project = await request.app.course_db.one(
        "SELECT * FROM projects WHERE id = ? AND owner_id = ?", (project_id, user["id"])
    )
    if project is None:
        raise NotFound("Project not found.")
    return project

@projects.get("/")
async def list_projects(request):
    user = await require_user(request)
    items = await request.app.course_db.all(
        "SELECT * FROM projects WHERE owner_id = ? ORDER BY id DESC", (user["id"],)
    )
    return {"data": items}

@projects.post("/")
async def create_project(request):
    await check_csrf(request)
    user = await require_user(request)
    data = await json_object(request)
    name = text(data, "name")
    description = text(data, "description", 1000, required=False)
    project_id = await request.app.course_db.execute(
        "INSERT INTO projects(owner_id, name, description) VALUES (?, ?, ?)",
        (user["id"], name, description),
    )
    return JSONResponse(
        {"data": await owned_project(request, project_id)}, status_code=201
    )

```

Commands and demonstration

```

python scripts/course_api_demo.py
python -m pytest -q tests/test_api.py -k project_crud

```

Expected result

The demo registers a temporary learner, creates a project through HTTP, and reads it back. The ownership test uses another account and verifies that the second user cannot read or alter the first user's project.

Common errors

Hiding a button does not enforce authorization. Never trust `owner_id` from JSON. Trailing slash matters for the collection URL: use `/api/projects/`. Empty or oversized names are rejected on the server even if HTML validation is bypassed.

Viewer exercise and wrap-up

Add a test for a name containing only spaces. Predict the status before running it. Then describe why a project owned by another user returns 404.

Ask the viewer to pause, try the exercise, and compare the result with the chapter's expected behavior.

06 | Tasks, status, dates, and filtering

Lesson outcome

Build the task workflow on top of project ownership.

Recording preparation

Target edited length: 20-25 minutes. This is a planning range, not a recording already made.

Before the take: Project creation and ownership must be working. Prepare one owned project and a second account.

Take 1 | Say, type, show

Say: A task belongs to a project. Every operation checks that the current user may access the parent project, even when the request only contains a task ID.

Type or explain on screen: `owned_task()`, `list_tasks()`, and `create_task()` in `modules/tasks/module.py`.

Show and verify: Create a task with a due date and demonstrate that another user cannot add a task to your project.

Take 2 | Say, type, show

Say: PATCH changes only supplied fields. Status is one of three values, and a date uses YYYY-MM-DD. These rules belong on the server as well as the form.

Type or explain on screen: `task_fields()`, `update_task()`, and `delete_task()`; type the optional list status filter.

Show and verify: Update a task to done, filter to done, reject an impossible date, and show the cascade test for project deletion.

Editing and chapter handoff

Editing note: Do not spend the take typing repetitive SQL punctuation silently. Explain the changed values and ownership decision.

End the chapter: Ask the learner to add an invalid-date regression case.

How it works

Tasks use `todo`, `doing`, and `done`. Creating a task checks its project first. Updating a task loads the task, then verifies its project ownership; knowing a task ID is not authorization. PATCH changes only supplied fields. Due dates use ISO YYYY-MM-DD or null.

A list request may filter by status, but SQL still includes the project ID. The API rejects unknown statuses before querying. Parameterized SQL safely handles titles containing quotation marks. Task deletion is explicit; project deletion cascades through the database.

Exact code reference for this take

File: `validation.py`

```
def task_fields(data, partial=False):
    result = {}
    if not partial or "title" in data:
        result["title"] = text(data, "title", 160)
    if not partial or "status" in data:
        status = data.get("status", "todo")
        if status not in ("todo", "doing", "done"):
            raise BadRequest("Status must be todo, doing, or done.")
        result["status"] = status
    if not partial or "due_date" in data:
        due = data.get("due_date") or None
        if due is not None:
            try:
                due = date.fromisoformat(due).isoformat()
            except (ValueError, TypeError):
                raise BadRequest("Use YYYY-MM-DD for a due date.")
        result["due_date"] = due
    if not result:
        raise BadRequest("Supply at least one editable field.")
    return result
```

Commands and demonstration

```
python -m pytest -q tests/test_api.py -k task
# Signed-in browser / authenticated HTTP client:
# GET /api/tasks/project/PROJECT_ID?status=done
```

Expected result

You can create a task, change its status, clear its due date, filter the project task list, and delete it. Invalid statuses and impossible dates receive error responses. The task module is included in full in the appendix.

Common errors

Do not filter all tasks first and authorize later. Passing an empty PATCH body is an error. Do not confuse a date-only field with an instant in a timezone. Client-side filtering improves presentation; API authorization remains mandatory.

Viewer exercise and wrap-up

Add a task with February 30 as its due date. Verify the rejection and add a regression assertion.

Ask the viewer to pause, try the exercise, and compare the result with the chapter's expected behavior.

07 | Test the backend before the frontend

Lesson outcome

Verify successful requests and denied requests using disposable databases.

Recording preparation

Target edited length: 15-20 minutes. This is a planning range, not a recording already made.

Before the take: Keep the API layer working before showing any customer interface code. Use disposable tests, never the recording database.

Take 1 | Say, type, show

Say: These tests make requests through the ASGI application. A temporary database gives each test an isolated place to create records.

Type or explain on screen: The application fixture and Account helper in tests/conftest.py. Explain how the helper carries the cookie and CSRF token.

Show and verify: Show the migration fixture and `create_app()` receiving temporary paths.

Take 2 | Say, type, show

Say: The ownership test registers a second user and calls the API directly. Hiding an Edit button would not pass this test if the backend were wrong.

Type or explain on screen: `test_project_crud_and_ownership()`, then one invalid task case.

Show and verify: Run focused tests first, then the suite appropriate to your current teaching checkout. The completed reference has additional Admin/CMS tests taught later.

Editing and chapter handoff

Editing note: Show failures intentionally only in a throwaway rehearsal checkout; restore the code and end the take with passing checks.

End the chapter: Point to backend-ready and explain that the frontend will consume these same endpoints.

How it works

The tests create the schema in a temporary directory and call `create_app()` with those paths. The `TestClient` wraps `HTTPX ASGITransport`, so requests exercise routing, middleware, validation, and handlers without opening a TCP port. The Account helper establishes a session and carries cookies and CSRF headers between requests.

Study one test line by line: arrange two accounts, create a project, exercise another user's access, and assert the response. Assertions should protect behavior that could break or leak information, not merely repeat implementation details.

Exact code reference for this take

File: tests/test_api.py

```
def test_project_crud_and_ownership(client, account):
    item = project(account)
    other = Account(client, "other@example.test")
    path = f"/api/projects/{item['id']}"
    for method in ["get", "put", "delete"]:
        assert (
            other.call(
                path, method, {"name": "Stolen"} if method != "get" else None
            ).status_code
            == 404
        )
    assert other.call("/api/projects/").json()["data"] == []
    assert (
        account.call(path, "put", {"name": "Renamed", "description": "Updated"}).json()[
            "data"
        ][
            "name"
        ]
        == "Renamed"
    )
    assert account.call(path, "delete", {}).status_code == 200
    assert account.call(path).status_code == 404
```

Commands and demonstration

```
python -m pytest -q
python -m pip check
```

Expected result

The verified full course suite has 43 passing cases, including parameterized inputs. Tests cover authentication, session expiry, ownership, task validation, body limits, Admin boundaries, and CMS publishing permissions. Counts may grow when you add exercises.

Common errors

Do not point tests at your development database. A shared cookie between two Account objects invalidates the ownership scenario. A test that asserts only 200 misses whether the returned records belong to the correct user.

Viewer exercise and wrap-up

Write one denied-request test of your own. Explain which user, input, or permission makes the request invalid.

Ask the viewer to pause, try the exercise, and compare the result with the chapter's expected behavior.

08 | First Teloce HTML screen

Lesson outcome

Connect the server to a browser SPA shell.

Recording preparation

Target edited length: 25-30 minutes. This is a planning range, not a recording already made.

Before the take: The backend must already work. Open `ui/app.html`, `ui/pages/Home.html`, `app.py`, and the module UI route declarations.

Take 1 | Say, type, show

Say: Our HTML component has a template, a TypeScript script, and a scoped style block. Flaxon serves the compiled shell; Teloce mounts pages inside the router view.

Type or explain on screen: The shell template and script, `app.use_teloce()`, and the explicit Python shell routes.

Show and verify: Open the compiled page and trace the router-view marker and module UI mapping.

Take 2 | Say, type, show

Say: We keep page styles inside the page. The compiler adds a component scope attribute so a header or button rule does not unexpectedly change another component.

Type or explain on screen: The shell style scoped block, then the Home component's `.hero`, `.eyebrow`, and `.button` rules. Explain the single `:global(body)` margin reset because body is outside the component.

Show and verify: Change the Home button colour, navigate away, and confirm another component's button is unchanged. Restore the final colour.

Take 3 | Say, type, show

Say: A parent owns its layout, and a child owns its form. Inherited font and colour can flow through the shell, but scoped selector rules do not become a global stylesheet.

Type or explain on screen: Show local style blocks in `Login`, `ProjectList`, `TaskForm`, and `TaskList`. Remove the old `app.css` reference when following an older checkout.

Show and verify: Confirm there is no `/assets/app.css` request. At mobile width, show that the shell navigation wraps and the forms fit.

Editing and chapter handoff

Editing note: Teach a few meaningful CSS rules live, then supply the rest of each scoped block in that component. Do not paste one giant global CSS file.

End the chapter: Ask viewers to explain the difference between inheritance and scoped selector matching.

How it works

Flaxon calls `app.use_teloce()` with the project root, UI directory, and MinifyJS option. The compiler builds `.html` components and serves the runtime and assets. `request.compile("app.html", {})` returns the compiled shell for known browser paths. The shell provides a router view; route pages mount inside it.

A component contains a template, a TypeScript script, and scoped style. Each page and child component owns its own CSS. The compiler adds a component attribute to elements and rewrites selectors to match that attribute, so parent selectors do not automatically style child internals. Font and colour may still inherit normally. The shell uses only a deliberate `:global(body)` margin reset because body is outside the component. There is no `public/app.css` or `stylesheets` argument in the completed factory. This course uses `.html`, not `.vel`. Module UI routes map the compiled page names to URL patterns. Flaxon can also render whole applications with Jinax; this project chooses Teloce for customer screens and server-rendered Admin for staff.

Exact code reference for this take

File: `ui/app.html`

```
<template>
  <div class="app-shell">
    <header><a href="/" data-teloce-link class="brand">Project Manager</a>
      <nav aria-label="Main navigation">
        <a href="/projects" data-teloce-link>Projects</a>
        <a href="/help" data-teloce-link>Help</a>
        <a href="/login" data-teloce-link>Account</a>
      </nav>
    </header>
    <main id="router-view" data-teloce-router-view></main>
    <footer>Built with Flaxon and Teloce. <a href="/admin/">Staff Admin</a></footer>
  </div>
</template>
<script lang="ts">export default {};</script>
<style scoped>
/* Only the document reset is global: body is outside this component. */
:global(body) {
  margin: 0;
}
.app-shell {
  min-height: 100vh;
  font-family: system-ui, sans-serif;
  color: #17243a;
  background: #f3f6fc;
}
header {
  display: flex;
  justify-content: space-between;
  gap: 1rem;
  align-items: center;
  padding: 1.2rem max(1rem, calc((100vw - 1080px) / 2));
  background: #17243a;
  color: white;
}
header a {
  color: white;
  text-decoration: none;
}
.brand {
  font-weight: 800;
```

```

    font-size: 1.3rem;
  }
  nav {
    display: flex;
    gap: 1rem;
    flex-wrap: wrap;
  }
  main {
    max-width: 1080px;
    margin: auto;
    min-height: 75vh;
    padding: 2rem 1rem;
  }
  footer {
    padding: 1.5rem;
    text-align: center;
    color: #526079;
  }
  footer a {
    color: #1d4ed8;
  }
  :focus-visible {
    outline: 3px solid #f59e0b;
    outline-offset: 3px;
  }
  @media (max-width: 520px) {
    header {
      flex-direction: column;

      align-items: flex-start;
    }
    main {
      padding: 1rem;
    }
  }
}
</style>

```

Commands and demonstration

```
python -m flaxon run app:app --reload
# Open http://127.0.0.1:8000/projects
```

Expected result

Navigation and the page mount appear. Before sign-in the project screen shows an understandable account message; it does not display private records. Unknown API paths still return an API 404 rather than the HTML shell.

Common errors

A blank router view may mean a missing module UI route, compiler error, or wrong compiled name. Do not write a catch-all server route that replaces every API error with HTML. Check the terminal and browser console first.

Viewer exercise and wrap-up

Identify the shell, one module page, and one child component. Explain which part Flaxon executes and which part runs in the browser.

Ask the viewer to pause, try the exercise, and compare the result with the chapter's expected behavior.

09 | Authentication UI and typed API helper

Lesson outcome

Submit account forms and share consistent request handling.

Recording preparation

Target edited length: 25-30 minutes. This is a planning range, not a recording already made.

Before the take: The auth API and SPA shell must run. Keep `ui/types.ts`, `ui/api.ts`, and `Login.html` open.

Take 1 | Say, type, show

Say: The interface knows the shape of a Session, but the server still validates the real request. Our helper loads the session, sends cookies and CSRF, and unwraps data.

Type or explain on screen: User and Session interfaces, `loadSession()`, then `api()`. Explain `response.ok` and token replacement on auth responses.

Show and verify: Show one successful response and one error without exposing a real session cookie on camera.

Take 2 | Say, type, show

Say: The account form tracks loading and errors, disables repeated submissions, and clears the password after success. Switching modes keeps the form DOM with `v-show`.

Type or explain on screen: `Login.html` template, `data()`, `mounted()`, `submit()`, `logout()`, and its scoped form styles.

Show and verify: Register in the browser, sign out, try a wrong password, then sign in. Show the status/alert messages and the keyboard labels.

Editing and chapter handoff

Editing note: Slow down when the helper cache changes after authentication. An old CSRF token is a common error worth explaining.

End the chapter: Have learners add explanatory form text without weakening backend validation.

How it works

TypeScript interfaces describe the JSON contract for users, projects, tasks, sessions, and articles. The API helper loads the session, includes cookies and the CSRF token, unwraps data, and surfaces server error messages. Authentication responses update the cached session after rotation.

The Login component uses `v-model`, guarded submission, loading/error state, and `v-show` when switching form mode. Keeping the form DOM intact avoids listener loss found during this project's browser testing. Never store the `HttpOnly` session token in `localStorage`. TypeScript transpilation removes types; this build is not a full static type checker.

Exact code reference for this take

File: ui/api.ts

```
import type { Session } from './types';

let session: Session | null = null;
export async function loadSession(): Promise<Session> {
  const response = await fetch('/api/auth/session', { credentials: 'same-origin' });
  const payload = await response.json();
  if (!response.ok) throw new Error(payload.error?.message || 'Could not load your session.');
```

```
  session = payload.data;
  return session!;
}

export async function api<T>(path: string, method = 'GET', body?: unknown): Promise<T> {
  if (!session) await loadSession();
  const response = await fetch(path, {
    method,
    credentials: 'same-origin',
    headers: { 'Content-Type': 'application/json', 'X-CSRF-Token': session!.csrf },
    body: body === undefined ? undefined : JSON.stringify(body),
  });
  const payload = await response.json();
  if (!response.ok) {
    if (response.status === 401 || response.status === 403) session = null;
    throw new Error(payload.error?.message || 'The request failed. Please try again.');
```

```
  }
  if (path.startsWith('/api/auth/')) session = payload.data;
  return payload.data as T;
}
```

Commands and demonstration

```
python -m flaxon run app:app --reload
# Open /login; register, sign out, then sign in.
```

Expected result

Submitting disables duplicate actions, validation errors are visible, and valid authentication changes the account state. The next write uses the rotated CSRF token rather than the anonymous token.

Common errors

A wrong relative import can prevent compilation. A 403 immediately after registration suggests cached old CSRF state. Fetch resolves even for HTTP errors: inspect response.ok rather than assuming every resolved promise succeeded.

Viewer exercise and wrap-up

Attempt an incorrect password and inspect the message. Add a short explanation next to the password field without weakening server validation.

Ask the viewer to pause, try the exercise, and compare the result with the chapter's expected behavior.

10 | Components, signals, and progress

Lesson outcome

Make project tasks reactive and reuse the task form and list.

Recording preparation

Target edited length: 30-35 minutes. This is a planning range, not a recording already made.

Before the take: An authenticated learner can create a project. Open the detail page, TaskForm.html, and TaskList.html.

Take 1 | Say, type, show

Say: The form emits an event; the parent decides how to save it. We keep requests in the page so the child components stay easy to understand.

Type or explain on screen: TaskForm and TaskList templates, props and emits, then the parent createTask(), changeStatus(), and removeTask() methods.

Show and verify: Create a task, change its status, and delete it. Show the child's scoped .card and button rules.

Take 2 | Say, type, show

Say: The task signal holds the complete collection. Computed progress reads that signal. The filtered list changes what we show, not the total we use for completion.

Type or explain on screen: signal([]), computed completion, setTasks(), and the mounted effect in ProjectDetails. Do not add signal imports to this compiled HTML component.

Show and verify: Create two tasks and complete one: progress is 50%. Filter to done and confirm it stays 50%.

Take 3 | Say, type, show

Say: An effect is work that belongs to the mounted screen. Stop it before unmount so an old screen does not keep reacting.

Type or explain on screen: beforeUnmount() and the mutation busy/error guard; type the local progress style.

Show and verify: Navigate away and back, then test an empty project at 0%. Run the task and browser checks.

Editing and chapter handoff

Editing note: Explain signal reads with parentheses and writes with .set(). Do not imply ordinary TypeScript helper files receive the compiler's automatic imports.

End the chapter: Ask learners to compute a remaining-task count from the full collection.

How it works

ProjectDetails loads a project and its tasks in parallel. TaskForm emits a create event; TaskList emits status and remove events. The parent owns requests and replaces the task list after successful writes, giving the UI one clear source of truth. Busy guards prevent duplicate submissions.

The task signal holds the collection. Computed completion returns zero for an empty collection and rounds the percentage of done tasks. An effect copies that value into component data for display and is stopped before unmount. The .html compiler automatically imports detected signal helpers; do not add redundant signal imports here. Ordinary .ts files do not receive the same automatic imports.

Exact code reference for this take

File: modules/projects/ui/pages/ProjectDetails/[id].html

```
// The compiler imports these signal helpers automatically for this .html component.
const taskState = signal([]);
const completion = computed(() => {
  const tasks = taskState();
  return tasks.length ? Math.round(tasks.filter(task => task.status === 'done').length * 100 /
    tasks.length) : 0;
});
```

Commands and demonstration

```
python -m pytest -q tests/test_api.py -k task_workflow
python scripts/browser_smoke.py
```

Expected result

Changing one task from todo to done updates progress. Filtering hides or shows tasks without changing the overall completion calculation. An empty project remains at 0%, not NaN.

Common errors

Calling a signal reads it; .set() replaces its value. Updating only the component array without calling setTasks would leave the separate signal stale. An effect that survives unmount can keep obsolete component state alive.

Viewer exercise and wrap-up

Add two tasks, complete one, and expect 50%. Filter to done and explain why project progress should still use the full task collection.

Ask the viewer to pause, try the exercise, and compare the result with the chapter's expected behavior.

11 | SPA routing with data-teloce-link

Lesson outcome

Support internal navigation, Back/Forward, and direct refresh.

Recording preparation

Target edited length: 15-20 minutes. This is a planning range, not a recording already made.

Before the take: At least one project and its detail screen must be working. Keep browser Network and a separate console ready.

Take 1 | Say, type, show

Say: data-teloce-link tells the browser router to handle an internal anchor. The page mounts in the shell rather than reloading the document.

Type or explain on screen: Internal anchors in app.html and ProjectList.html; show the module's /projects/:id mapping and the id prop.

Show and verify: Set window.courseNavigationMarker = 42, click a project, and read the marker again. Back and Forward should preserve the document.

Take 2 | Say, type, show

Say: A refresh starts at the server. Flaxon must return the shell for the browser URL before Teloce can mount the component and fetch its data.

Type or explain on screen: The explicit /projects/<int:project_id> shell route, then the Help shell routes. Explain why Admin links are ordinary anchors.

Show and verify: Refresh a nested project URL and request an unknown /api path to show that API 404 responses are preserved.

Editing and chapter handoff

Editing note: Do not hide the refresh check; it is the difference between click-only routing and a usable SPA deployment.

End the chapter: Ask learners to add a Help link and verify click, Back, and refresh.

How it works

Three parts cooperate. First, module ui_routes register /projects and /projects/:id in the browser. Second, data-teloce-router-view marks the shell mount point. Third, data-teloce-link on internal anchors lets Teloce navigate without reloading the document. Dynamic IDs become component props.

Python shell routes are still necessary for a direct visit or refresh at /projects/42. The server returns the shell, then Teloce mounts the page and requests its authorized JSON data. Staff Admin is a separate interface; its anchor intentionally navigates normally. After deleting a project, the component uses the existing router's navigate method.

Exact code reference for this take

File: app.py

```
# Explicit shell routes preserve real 404 responses for unknown API paths.
@app.get("/")
@app.get("/login")
@app.get("/projects")
@app.get("/projects/<int:project_id>")
@app.get("/help")
@app.get("/help/<slug>")
async def spa(request: Request, project_id=None, slug=None):
    return await request.compile("app.html", {})
```

Commands and demonstration

```
python scripts/browser_smoke.py
# Open /projects, select a project, use Back and Forward.
# Refresh the nested /projects/ID URL.
```

Expected result

Internal links keep the current document. Back/Forward mounts the corresponding page. A direct nested URL renders its screen after authentication, while an invalid API URL stays 404.

Common errors

A working click but failed refresh indicates missing Python shell routes. A full reload on an internal link indicates a missing data-teloce-link marker. Do not add the marker to unrelated Admin or external URLs. Route patterns and component props must agree.

Viewer exercise and wrap-up

Add a Help navigation link using the marker. Test click, Back, and direct refresh, then describe the server and browser work in each case.

Ask the viewer to pause, try the exercise, and compare the result with the chapter's expected behavior.

12 | Staff Admin and model adapters

Lesson outcome

Manage customer projects and tasks through separate staff accounts.

Recording preparation

Target edited length: 20-25 minutes. This is a planning range, not a recording already made.

Before the take: Customer APIs work. Open `backoffice.py` and `management.py`. Use only disposable staff credentials during the recording.

Take 1 | Say, type, show

Say: Customers and staff are separate account systems. Registering a customer never creates an administrator. We bootstrap staff interactively, with no default password.

Type or explain on screen: AdminDashboard configuration with `strict_permissions`, then the `setup-admin` command.

Show and verify: Run `setup-admin` with the password entry off-screen. Sign into `/admin/login` and show the model list.

Take 2 | Say, type, show

Say: The Admin adapter reuses the same database as the customer API. Staff capabilities decide who can read or change all registered records; this is not a customer tenant portal.

Type or explain on screen: ProjectAdmin `get_instances()`, `get_instance()`, `update_instance()`, and registration fields. Explain why `create` is disabled here.

Show and verify: Edit a project as authorized staff and read it through its owner account. Create a read-only staff role and show that changes are denied.

Editing and chapter handoff

Editing note: Hide passwords and sensitive account data. Show exact permission keys from `docs/security.md` rather than implying a role name grants everything.

End the chapter: Recap the account boundary and introduce editorial permissions.

How it works

AdminDashboard registers a persistent staff store with `strict_permissions` enabled. Domain registration and Admin registration are separate. `management.py` `setup-admin` prompts for credentials without putting a password in Git or the command line.

The ProjectAdmin and TaskAdmin adapters reuse the same domain database as the APIs. Staff can view and change permitted records; creating records is deliberately left to the customer workflow so ownership is established correctly. Permissioned staff may access all registered records, so these adapters are not a tenant-restricted customer portal. Grant narrow capabilities and reserve superuser for trusted administrators.

Exact code reference for this take

File: backoffice.py

```
admin = AdminDashboard(
    app,
    registry=Registry(),
    users=[],
    storage_path=str(admin_path),
    upload_dir=str(uploads_path),
    strict_permissions=True,
    microservices=False,
    config=AdminConfig(site_title="Project Manager Operations"),
)
```

Commands and demonstration

```
python management.py setup-admin
python -m flaxon run app:app --reload
# Open /admin/login, then /admin/.
```

Expected result

A public account cannot sign into staff Admin. A staff administrator can see project and task model lists. Changes through an adapter persist in the same app.sqlite3 read by the customer API. Follow docs/security.md for exact role keys.

Common errors

Role names alone do not grant every custom model capability. Use project.view_project and task.view_task for readers, then specific change keys for operators. The standalone bootstrap command creates staff data but the server must still configure AdminDashboard.

Viewer exercise and wrap-up

Create a read-only staff group and verify an attempted edit is denied. Explain why hiding the Edit button alone is insufficient.

Ask the viewer to pause, try the exercise, and compare the result with the chapter's expected behavior.

13 | CMS help content and publishing

Lesson outcome

Publish sanitized help articles while preserving editorial boundaries.

Recording preparation

Target edited length: 20-25 minutes. This is a planning range, not a recording already made.

Before the take: Staff Admin runs with the patched course build. Open the help content type, public content module, Help.html, and Article/[slug].html.

Take 1 | Say, type, show

Say: The CMS stores editorial records. Public readers receive only published articles and a small whitelist of fields; they never use the staff editing API.

Type or explain on screen: The help_article ContentType, published_articles(), the list endpoint, and the slug endpoint.

Show and verify: Create a draft and show that it is absent from the Help page. Publish with a permitted account and load it in the SPA.

Take 2 | Say, type, show

Say: Rendering HTML is different from displaying a string. This field is sanitized rich text; we do not render arbitrary user input as HTML.

Type or explain on screen: Help and Article templates, v-html on the sanitized body, and each page's scoped styles.

Show and verify: Show the rendered article and the browser-safe body. Explain that injected HTML does not automatically receive component scope attributes.

Take 3 | Say, type, show

Say: Draft editors and publishers have different rights. Editing already published content or restoring a protected revision also needs the required publication permissions.

Type or explain on screen: Show the CMS security patch notes and one permission regression test; do not rewrite the entire framework patch during the lesson.

Show and verify: Run the CMS tests and demonstrate that a draft-only editor cannot publish.

Editing and chapter handoff

Editing note: If demonstrating seed, run it only after creating a customer account and restart the process. Do not seed production on every deployment.

End the chapter: Ask learners to prove that a draft is not exposed publicly.

How it works

The CMS receives the Admin authentication backend. `help_article` has title, summary, and rich-text body fields, with draft and published statuses. Public endpoints expose only published records and a small field whitelist. The Article component uses v-html only for rich text sanitized by nh3. Ordinary task/user text remains escaped.

The bundled framework patch makes CMS without auth deny access by default. Creating, editing, importing, restoring, and acting on protected publication states require publishing rights. Editing a published article also requires publication permission; a draft-only editor must not silently change live content. These are targeted fixes, not a complete security audit.

Exact code reference for this take

File: `backoffice.py`

```
cms = CMS(app, auth=admin.auth)
cms.register(
    ContentType(
        "help_article",
        label="Help article",
        label_plural="Help articles",
        fields=[
            CMSField("title", required=True),
            CMSField("summary", type="textarea"),
            CMSField("body", type="richtext"),
        ],
        statuses=["draft", "published"],
        list_display=["title", "status"],
    )
)
```

Commands and demonstration

```
python management.py seed
# Restart the server after seeding.
python -m pytest -q tests/test cms_security.py
# Browse /admin/cms/ and /help.
```

Expected result

Only published help content appears publicly. Draft edits and publication rights are distinct. Help editors need model view/add/change permissions; publishers additionally need `cms.publish_content` and, when restoring revisions, `cms.restore_revision`.

Common errors

Do not expose the authenticated CMS editing API as a public content feed. Do not trust raw HTML from an arbitrary field. Seed only after a domain account exists and restart afterward because the CMS keeps in-process content state.

Viewer exercise and wrap-up

Create a draft and verify it is absent from `/help`. Have an authorized publisher publish it, then verify the article appears.

Ask the viewer to pause, try the exercise, and compare the result with the chapter's expected behavior.

14 | Full-stack verification and accessibility

Lesson outcome

Check real browser behavior in development and production modes.

Recording preparation

Target edited length: 15-20 minutes. This is a planning range, not a recording already made.

Before the take: The completed app and Playwright Chromium must be installed. Rehearse the browser script before recording its terminal output.

Take 1 | Say, type, show

Say: API tests cannot prove that a real form listener, SPA link, or mobile layout works. Our browser checks use fresh data and exercise the actual compiled interface.

Type or explain on screen: Explain the server setup and desktop/mobile loop in `scripts/browser_smoke.py`; add one behavioral assertion as a teaching example.

Show and verify: Run API checks and the development browser smoke test. Show the scoped form controls and a 390-pixel viewport.

Take 2 | Say, type, show

Say: We also test the production build, because minified code should preserve the same behavior. Accessibility is part of the working interface, not a final decoration.

Type or explain on screen: Point to the production switch, status/alert roles, form labels, focus-visible styles, and busy guards.

Show and verify: Run the production browser smoke test. Complete a short keyboard-only workflow and show a visible focus ring.

Editing and chapter handoff

Editing note: Cut repetitive automated interactions if they are too fast to teach; keep the purpose, command, and passing result.

End the chapter: List the checks that must pass before the deployment take.

How it works

API tests cannot prove that a compiled form listener works, a route preserves the document, or a layout fits mobile. The browser smoke script uses disposable data and Chromium at desktop and mobile sizes. It checks account forms, project/task workflows, progress, staff content, navigation, history, refresh, and logout.

Loading messages use status semantics; errors use alerts; labels name form controls. Busy state is visible and buttons are disabled during work. Verify keyboard focus and contrast as you change visual styles. Production smoke checks the minified interface locally, not the actual Render infrastructure.

Exact code reference for this take

File: scripts/build_ui.py

```
"""Build production Teloce assets without touching the live database or Render disk."""

import os
import secrets
import sys
import tempfile
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
with tempfile.TemporaryDirectory() as build_data:
    os.environ["DATA_DIR"] = build_data
    os.environ["FLAXON_DEBUG"] = "0"
    os.environ.setdefault("FLAXON_SECRET_KEY", secrets.token_urlsafe(48))
    from app import app

    result = app.telocce.build()
    print(f"Built {result.get('compiled', 0)} Teloce components with MinifyJS.")
```

Commands and demonstration

```
python -m playwright install chromium
python -m pytest -q
python scripts/browser_smoke.py
python scripts/browser_smoke.py --production
python -m pip check
```

Expected result

The full suite and both browser modes pass. Browser tests create their own databases rather than modifying your learner records. Desktop and 390-pixel mobile layouts support the same workflow.

Common errors

Install Chromium before running Playwright. A port already in use can prevent the test server starting. Local Chromium treats loopback as trustworthy; an actual deployed site must use HTTPS. Do not treat passing local smoke tests as proof of cloud deployment.

Viewer exercise and wrap-up

Complete the workflow using only the keyboard. Record one accessibility improvement and retest it at mobile width.

Ask the viewer to pause, try the exercise, and compare the result with the chapter's expected behavior.

15 | Production build and Render deployment

Lesson outcome

Prepare one persistent instance with secrets, HTTPS, and verified data survival.

Recording preparation

Target edited length: 20-30 minutes. This is a planning range, not a recording already made.

Before the take: Keep the local production tests green. A real deployment take requires your own Render service access; do not say deployment succeeded until you verify it there.

Take 1 | Say, type, show

Say: The compiled interface uses MinifyJS. Component CSS is bundled from style scoped blocks; there is no separate app.css file to copy to a server.

Type or explain on screen: Explain scripts/build_ui.py and the build command in render.yaml.

Show and verify: Run the production build and show the component count and browser verification result.

Take 2 | Say, type, show

Say: SQLite needs a persistent disk. This course uses one instance and one worker, with both application and Admin databases under /var/data.

Type or explain on screen: Render disk, DATA_DIR, runtime migration/start command, PUBLIC_ORIGIN, DEBUG, and generated secret settings.

Show and verify: In your Render account, configure the actual HTTPS origin and bootstrap staff with the Shell. Keep secrets off-camera.

Take 3 | Say, type, show

Say: A successful service screen is not enough. We need a real workflow, denied requests, and data that survives a restart.

Type or explain on screen: Use the verification list in docs/render.md and write down a backup/restore procedure for both SQLite files.

Show and verify: Create a project/task, publish help, refresh a nested route, sign out, confirm API denial, restart, and verify persistence. If you have not deployed, explicitly present this as preparation.

Editing and chapter handoff

Editing note: Record the cloud verification after it actually works; do not splice a local test into a claim of live HTTPS verification.

End the chapter: End with the completed application, source/checkpoint links, and one extension exercise. Thank viewers in your own voice.

How it works

MinifyJS runs through the Teloce production build. Runtime dependencies are pinned. The Render Blueprint uses one Python service, one worker, and a persistent disk mounted at /var/data. Both app.sqlite3 and admin.sqlite3 must survive restarts. SQLite migrations run in the start command because the live disk is available at runtime rather than build time.

Set FLAXON_DEBUG=0, a random secret of at least 32 characters, and the exact HTTPS PUBLIC_ORIGIN. Production Host checks and browser Origin checks depend on that URL. Create staff interactively in the service Shell. This book describes the repository's prepared configuration; it has not been deployed to your Render account. Verify current service settings with the official references listed in docs/render.md.

Exact code reference for this take

File: render.yaml

```
services:
- type: web
  name: flaxon-project-manager
  runtime: python
  plan: starter
  buildCommand: python scripts/verify_vendor.py && pip install -r requirements.txt && python
    scripts/build_ui.py
  startCommand: python management.py migrate && python -m uvicorn app:app --host 0.0.0.0
    --port $PORT --workers 1
  healthCheckPath: /health/course
  disk:
    name: project-manager-data
    mountPath: /var/data
    sizeGB: 1
  envVars:
    - key: FLAXON_DEBUG
      value: "0"
    - key: FLAXON_SECRET_KEY
      generateValue: true
    - key: DATA_DIR
      value: /var/data
    - key: PUBLIC_ORIGIN
      sync: false
```

Commands and demonstration

```
python scripts/verify_vendor.py
python scripts/build_ui.py
python -m pip check
# In Render Shell after deployment:
python management.py setup-admin
```

Expected result

After configuring the service, check /health/course, register, create a project/task, publish help, refresh a nested route, sign out, and restart the service. Data must persist. Secure and HttpOnly cookies must appear over HTTPS; debug traces must be absent.

Common errors

An ephemeral filesystem loses SQLite data. More workers or instances need shared storage and coordination; do not increase them with this design. A changed domain requires PUBLIC_ORIGIN to change too. Do not seed automatically at every deployment.

Viewer exercise and wrap-up

Write a recovery checklist and back up both databases using SQLite's online backup API or a quiesced copy. Restore into a test environment and verify one project and one help article.

Ask the viewer to pause, try the exercise, and compare the result with the chapter's expected behavior.

Appendix A | Complete application source

These files are printed in full so you can follow the lesson without reconstructing missing handlers. The repository additionally contains the pinned transitive lock, vendored wheels, welcome starter reference, test fixtures, regression suites, and browser smoke script. Clone it to obtain binary dependencies and all tests. PDF code lines may wrap for print; copy exact code from Markdown or repository files.

settings.py

File: settings.py

```
"""Environment settings; both the application and management commands use these."""

import os
from pathlib import Path
from dotenv import load_dotenv

ROOT = Path(__file__).resolve().parent
load_dotenv(ROOT / ".env")
PROJECT_NAME = "Project Manager"
DATA_DIR = Path(os.getenv("DATA_DIR", str(ROOT / "data")))
DATABASE_PATH = DATA_DIR / "app.sqlite3"
ADMIN_DATABASE_PATH = DATA_DIR / "admin.sqlite3"
DEBUG = os.getenv("FLAXON_DEBUG", "1") == "1"
PUBLIC_ORIGIN = os.getenv("PUBLIC_ORIGIN", "http://127.0.0.1:8000").rstrip("/")
```

app.py

File: app.py

```
"""Compose the CLI starter's modules, staff backoffice, and Teloce SPA."""

import os
from pathlib import Path
from urllib.parse import urlsplit
from flaxon.middleware import BodyLimitMiddleware, TrustedHostsMiddleware
from argon2 import PasswordHasher
from flaxon import Flaxon, Request
from flaxon.http import JSONResponse
from flaxon.exceptions import BadRequest
from database import Database
from backoffice import configure_backoffice
from modules.auth.module import auth
from modules.projects.module import projects
from modules.tasks.module import tasks
from modules.content.module import content
from modules.welcome.module import welcome
from settings import (
    ROOT,
    DATA_DIR,
    DATABASE_PATH,
    ADMIN_DATABASE_PATH,
    DEBUG,
    PUBLIC_ORIGIN,
)

def create_app(database_path=None, admin_path=None, debug=None):
```

```

debug = DEBUG if debug is None else debug
secret_key = os.getenv("FLAXON_SECRET_KEY", "")
if not debug and len(secret_key) < 32:
    raise ValueError(
        "Set FLAXON_SECRET_KEY to at least 32 random characters in production."
    )

app = Flaxon(
    "project-manager",
    debug=debug,
    openapi=True,
    config={"SECRET_KEY": secret_key or None},
)
app.add_middleware(BodyLimitMiddleware, max_size=256 * 1024)
if not debug:
    app.add_middleware(
        TrustedHostsMiddleware, allowed_hosts=[urlsplit(PUBLIC_ORIGIN).hostname]
    )
app.course_db = Database(database_path or DATABASE_PATH)
app.public_origin = PUBLIC_ORIGIN
app.hasher = PasswordHasher()
app.dummy_password_hash = app.hasher.hash("dummy-password-for-timing-only")
for module, prefix in [
    (auth, "/api/auth"),
    (projects, "/api/projects"),
    (tasks, "/api/tasks"),
    (content, "/api/content"),
    (welcome, "/api/welcome"),
]:
    app.mount_module(module, prefix=prefix)
configure_backoffice(
    app,
    admin_path or ADMIN_DATABASE_PATH,
    Path(admin_path or ADMIN_DATABASE_PATH).parent / "uploads",
)
app.use_teloc(
    project_root=ROOT,
    ui_dir="ui",
    title="Project Manager",
    options={"minifier": "minifyjs"},
)

@app.get("/health/course")
async def health():
    await app.course_db.one("SELECT 1 FROM users LIMIT 1")
    return {"data": {"status": "ok"}}

# Explicit shell routes preserve real 404 responses for unknown API paths.
@app.get("/")
@app.get("/login")
@app.get("/projects")
@app.get("/projects/<int:project_id>")
@app.get("/help")
@app.get("/help/<slug>")
async def spa(request: Request, project_id=None, slug=None):
    return await request.compile("app.html", {})

return app

```

```
app = create_app()
```

database.py

File: database.py

```
"""Small SQLite repository: parameterized SQL, explicit transactions, no global connection."""

import asyncio
import sqlite3
from contextlib import contextmanager
from pathlib import Path

class Database:
    def __init__(self, path):
        self.path = Path(path)
        self.path.parent.mkdir(parents=True, exist_ok=True)

    @contextmanager
    def connection(self):
        connection = sqlite3.connect(self.path, timeout=10)
        connection.row_factory = sqlite3.Row
        connection.execute("PRAGMA foreign_keys = ON")
        try:
            with connection:
                yield connection
        finally:
            connection.close()

    async def all(self, sql, parameters=()):
        def read():
            with self.connection() as connection:
                return [
                    dict(row) for row in connection.execute(sql, parameters).fetchall()
                ]

        return await asyncio.to_thread(read)

    async def one(self, sql, parameters=()):
        rows = await self.all(sql, parameters)
        return rows[0] if rows else None

    async def execute(self, sql, parameters=()):
        def run():
            with self.connection() as connection:
                cursor = connection.execute(sql, parameters)
                return cursor.lastrowid

        return await asyncio.to_thread(run)
```

security.py

File: security.py

```
"""Opaque cookie sessions and session-bound CSRF; account ownership stays on the server."""

import asyncio
import hashlib
import hmac
import secrets
import time
from flaxon.exceptions import Forbidden, Unauthorized, TooManyRequests
from flaxon.http import JSONResponse
from flaxon.http.cookies import Cookie
```

```

COOKIE_NAME = "project_session"
SESSION_SECONDS = 8 * 60 * 60

def digest(value):
    return hashlib.sha256(value.encode()).hexdigest()

async def read_session(request):
    token = request.cookies.get(COOKIE_NAME, "")
    if not token:
        return None
    return await request.app.course_db.one(
        "SELECT * FROM sessions WHERE token_hash = ? AND expires_at > ?",
        (digest(token), int(time.time()))
    )

async def require_user(request):
    session = await read_session(request)
    if not session or session["user_id"] is None:
        raise Unauthorized("Please sign in.")
    user = await request.app.course_db.one(
        "SELECT id, name, email FROM users WHERE id = ?", (session["user_id"],)
    )
    if user is None:
        raise Unauthorized("Please sign in.")
    return user

async def check_csrf(request):
    origin = request.headers.get("origin")
    if origin and origin != request.app.public_origin:
        raise Forbidden("This request came from an untrusted origin.")
    if "application/json" not in request.headers.get("content-type", "").lower():
        raise Forbidden("Send JSON with a CSRF token.")
    session = await read_session(request)
    supplied = request.headers.get("x-csrf-token", "")
    if (
        not session
        or not supplied
        or not hmac.compare_digest(session["csrf"], supplied)
    ):
        raise Forbidden("Your session expired. Reload the page and try again.")

async def session_response(request, user=None, status=200):
    old = request.cookies.get(COOKIE_NAME, "")
    if old:
        await request.app.course_db.execute(
            "DELETE FROM sessions WHERE token_hash = ?", (digest(old),)
        )
    token, csrf = secrets.token_urlsafe(32), secrets.token_urlsafe(32)
    await request.app.course_db.execute(
        "DELETE FROM sessions WHERE expires_at <= ?", (int(time.time()),)
    )
    await request.app.course_db.execute(
        "INSERT INTO sessions(token_hash, user_id, csrf, expires_at) VALUES (?, ?, ?, ?)",
        (
            digest(token),
            user["id"] if user else None,
            csrf,
            int(time.time() + SESSION_SECONDS)
        )
    )

```

```

        int(time.time()) + SESSION_SECONDS,
    ),
)
cookie = Cookie(
    COOKIE_NAME,
    token,
    path="/",
    httponly=True,
    samesite="Lax",
    secure=not request.app.debug,
    max_age=SESSION_SECONDS,
)
response = JSONResponse(
    {"data": {"user": user, "csrf": csrf}},
    status_code=status,
    headers={"cache-control": "no-store"},
)
response.headers.add("set-cookie", cookie.to_header())
return response

async def limit_auth_attempts(request, email):
    """A durable, single-instance limiter; production proxies also limit the auth routes."""
    peer = request.scope.get("client") or ("unknown", 0)
    key = digest(str(peer[0]) + ":" + email)
    now = int(time.time())

    def count():
        with request.app.course_db.connection() as connection:
            connection.execute(
                "DELETE FROM auth_attempts WHERE started_at < ?", (now - 900,)
            )
            connection.execute(
                "INSERT INTO auth_attempts(key, attempts, started_at) VALUES (?, 1, ?) ON"
            )
            CONFLICT(key) DO UPDATE SET attempts = attempts + 1",
                (key, now),
            )
            return connection.execute(
                "SELECT attempts FROM auth_attempts WHERE key = ?", (key,)
            ).fetchone()[0]

    if await asyncio.to_thread(count) > 10:
        raise TooManyRequests("Too many attempts. Try again in 15 minutes.")

```

validation.py

File: validation.py

```

"""Validate every write on the server, even when the browser already validated it."""

import re
from datetime import date
from flaxon.exceptions import BadRequest

async def json_object(request):
    try:
        data = await request.json()
    except (ValueError, UnicodeDecodeError):
        raise BadRequest("Send a valid JSON object.")
    if not isinstance(data, dict):
        raise BadRequest("Send a JSON object.")
    return data

```

```

def text(data, key, maximum=120, required=True):
    value = data.get(key, "")
    if not isinstance(value, str):
        raise BadRequest(f"{key} must be text.")
    value = value.strip()
    if (required and not value) or len(value) > maximum:
        raise BadRequest(
            f"{key} must contain 1 to {maximum} characters."
            if required
            else f"{key} must contain at most {maximum} characters."
        )
    return value

def credentials(data):
    email = text(data, "email", 254).lower()
    if not re.fullmatch(r"^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)", email):

        raise BadRequest("Enter a valid email address.")
    password = data.get("password")
    if not isinstance(password, str) or not 12 <= len(password) <= 128:
        raise BadRequest("Use a password with 12 to 128 characters.")
    return email, password

def task_fields(data, partial=False):
    result = {}
    if not partial or "title" in data:
        result["title"] = text(data, "title", 160)
    if not partial or "status" in data:
        status = data.get("status", "todo")
        if status not in ("todo", "doing", "done"):
            raise BadRequest("Status must be todo, doing, or done.")
        result["status"] = status
    if not partial or "due_date" in data:
        due = data.get("due_date") or None
        if due is not None:
            try:
                due = date.fromisoformat(due).isoformat()
            except (ValueError, TypeError):
                raise BadRequest("Use YYYY-MM-DD for a due date.")
        result["due_date"] = due
    if not result:
        raise BadRequest("Supply at least one editable field.")
    return result

```

management.py

File: management.py

```

"""Project-local administration. Run --help to see available commands."""

import argparse
import asyncio
import getpass

from flaxon.admin.services import AdminAuth, AdminStore
from flaxon.database.adapters.sqlite import SQLiteAdapter
from flaxon.database.manager import DatabaseManager
from flaxon.database.migrations import MigrationRunner
from settings import ROOT, DATA_DIR, DATABASE_PATH, ADMIN_DATABASE_PATH

async def migrate(status=False):

```

```

DATA_DIR.mkdir(parents=True, exist_ok=True)
db = DatabaseManager(SQLiteAdapter(database=str(DATABASE_PATH)))
await db.initialize()
try:
    runner = MigrationRunner(db, migration_dir=str(ROOT / "migrations"))
    if status:
        report = await runner.status()
        print(
            f"{report['applied_count']} applied, {report['pending_count']} pending"
        )
    else:
        applied = await runner.migrate()
        print(f"Applied {len(applied)} migration(s).")
finally:
    await db.close()

def setup_admin(username=None):
    DATA_DIR.mkdir(parents=True, exist_ok=True)
    store = AdminStore(str(ADMIN_DATABASE_PATH))

    username = (username or input("Administrator username: ")).strip()
    if not username:
        raise ValueError("A username is required")
    if store.get("users", username) is not None:
        raise ValueError(
            "That administrator already exists; use the admin interface to manage it"
        )
    password = getpass.getpass("Password: ")
    if password != getpass.getpass("Confirm password: "):
        raise ValueError("Passwords do not match")
    auth = AdminAuth(users=[], store=store, strict_permissions=True)
    record = auth.add_user(
        {
            "username": username,
            "password": password,
            "roles": ["administrator"],
            "permissions": ["admin.superuser"],
        }
    )

def create(existing):
    if existing:
        raise ValueError("That administrator already exists")
    existing.update(record)

store.mutate("users", username, create, default={})
print(f"Administrator '{username}' created. Sign in at /admin/login.")

def main(argv=None):
    parser = argparse.ArgumentParser(description="Manage your Flaxon application")
    commands = parser.add_subparsers(dest="command", required=True)
    migration = commands.add_parser(
        "migrate", help="Apply the project's database migrations"
    )
    migration.add_argument("--status", action="store_true")
    admin = commands.add_parser(
        "setup-admin",
        aliases=["createsuperuser"],
        help="Create an administrator securely",
    )
    admin.add_argument("--username")

```

```

commands.add_parser(
    "seed",
    help="Create sample projects and published help content; requires a registered
account",
)
args = parser.parse_args(argv)
try:
    if args.command == "migrate":
        asyncio.run(migrate(args.status))
    elif args.command == "seed":
        from seed import seed

        asyncio.run(seed())
    else:
        setup_admin(args.username)
except (ValueError, OSError) as error:
    parser.exit(1, f"Error: {error}\n")
return 0

```

```

if __name__ == "__main__":
    raise SystemExit(main())

```

migrations/0001_initial.json

File: migrations/0001_initial.json

```

{
  "version": "0001",
  "name": "users_projects_tasks",
  "up": "CREATE TABLE users (id INTEGER PRIMARY KEY, name TEXT NOT NULL, email TEXT NOT NULL
UNIQUE COLLATE NOCASE, password_hash TEXT NOT NULL);\nCREATE TABLE projects (id INTEGER
PRIMARY KEY, owner_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE, name TEXT
NOT NULL, description TEXT NOT NULL DEFAULT '', created_at TEXT NOT NULL DEFAULT
CURRENT_TIMESTAMP);\nCREATE INDEX projects_owner ON projects(owner_id);\nCREATE TABLE
tasks (id INTEGER PRIMARY KEY, project_id INTEGER NOT NULL REFERENCES projects(id) ON
DELETE CASCADE, title TEXT NOT NULL, status TEXT NOT NULL DEFAULT 'todo' CHECK(status IN
('todo','doing','done')), due_date TEXT, created_at TEXT NOT NULL DEFAULT
CURRENT_TIMESTAMP);\nCREATE INDEX tasks_project ON tasks(project_id);\nCREATE TABLE
sessions(token_hash TEXT PRIMARY KEY, user_id INTEGER REFERENCES users(id) ON DELETE
CASCADE, csrf TEXT NOT NULL, expires_at INTEGER NOT NULL);\nCREATE TABLE auth_attempts(key
TEXT PRIMARY KEY, attempts INTEGER NOT NULL, started_at INTEGER NOT NULL);",
  "down": "DROP TABLE auth_attempts; DROP TABLE sessions; DROP TABLE tasks; DROP TABLE
projects; DROP TABLE users;"
}

```

modules/auth/module.py

File: modules/auth/module.py

```

"""Registration and cookie authentication, separate from staff Admin accounts."""

import asyncio
import sqlite3
from pathlib import Path
from argon2.exceptions import VerifyMismatchError, VerificationError
from flaxon.modules import FlaxonModule
from flaxon.exceptions import Conflict, Unauthorized
from flaxon.http import JSONResponse
from security import (
    check_csrf,
    read_session,
    require_user,
    session_response,

```

```

        limit_auth_attempts,
    )
from validation import json_object, text, credentials

auth = FlaxonModule(
    "auth", ui_dir=Path(__file__).parent / "ui", ui_routes={"Login.js": "/login"}
)

@auth.get("/session")
async def session(request):
    current = await read_session(request)
    if current:
        user = await require_user(request) if current["user_id"] else None
        return JSONResponse(
            {"data": {"user": user, "csrf": current["csrf"]}},
            headers={"cache-control": "no-store"},
        )
    return await session_response(request)

@auth.post("/register")
async def register(request):
    await check_csrf(request)
    data = await json_object(request)
    email, password = credentials(data)
    name = text(data, "name", 80)
    await limit_auth_attempts(request, email)
    password_hash = await asyncio.to_thread(request.app.hasher.hash, password)
    try:
        user_id = await request.app.course_db.execute(
            "INSERT INTO users(name, email, password_hash) VALUES (?, ?, ?)",
            (name, email, password_hash),
        )
    except sqlite3.IntegrityError:
        raise Conflict("Unable to create this account. Try signing in.")
    return await session_response(
        request, {"id": user_id, "name": name, "email": email}, status=201
    )

@auth.post("/login")
async def login(request):
    await check_csrf(request)
    email, password = credentials(await json_object(request))
    await limit_auth_attempts(request, email)
    user = await request.app.course_db.one(
        "SELECT * FROM users WHERE email = ?", (email,)
    )
    password_hash = user["password_hash"] if user else request.app.dummy_password_hash
    try:
        await asyncio.to_thread(request.app.hasher.verify, password_hash, password)
    except (VerifyMismatchError, VerificationError):
        raise Unauthorized("Email or password is incorrect.")

    if user is None:
        raise Unauthorized("Email or password is incorrect.")
    public_user = {key: user[key] for key in ("id", "name", "email")}
    return await session_response(request, public_user)

@auth.post("/logout")
async def logout(request):
    await check_csrf(request)
    return await session_response(request)

```

modules/projects/module.py

File: modules/projects/module.py

```
"""Project APIs always constrain queries by the signed-in user's id."""

from pathlib import Path
from flaxon.modules import FlaxonModule
from flaxon.http import JSONResponse
from flaxon.exceptions import NotFound
from security import require_user, check_csrf
from validation import json_object, text

projects = FlaxonModule(
    "projects",
    ui_dir=Path(__file__).parent / "ui",
    ui_routes={
        "ProjectList.js": "/projects",
        "ProjectDetails/[id].js": "/projects/:id",
    },
)

async def owned_project(request, project_id):
    user = await require_user(request)
    project = await request.app.course_db.one(
        "SELECT * FROM projects WHERE id = ? AND owner_id = ?", (project_id, user["id"])
    )
    if project is None:
        raise NotFound("Project not found.")
    return project

@projects.get("/")
async def list_projects(request):
    user = await require_user(request)
    items = await request.app.course_db.all(
        "SELECT * FROM projects WHERE owner_id = ? ORDER BY id DESC", (user["id"],)
    )
    return {"data": items}

@projects.post("/")
async def create_project(request):
    await check_csrf(request)
    user = await require_user(request)
    data = await json_object(request)
    name = text(data, "name")
    description = text(data, "description", 1000, required=False)
    project_id = await request.app.course_db.execute(
        "INSERT INTO projects(owner_id, name, description) VALUES (?, ?, ?)",
        (user["id"], name, description),
    )
    return JSONResponse(
        {"data": await owned_project(request, project_id)}, status_code=201
    )

@projects.get("/<int:project_id>")
async def get_project(request, project_id):
    return {"data": await owned_project(request, project_id)}

@projects.put("/<int:project_id>")
```

```

async def update_project(request, project_id):
    await check_csrf(request)
    project = await owned_project(request, project_id)
    data = await json_object(request)
    name = text(data, "name")
    description = text(data, "description", 1000, required=False)
    await request.app.course_db.execute(
        "UPDATE projects SET name = ?, description = ? WHERE id = ? AND owner_id = ?",

        (name, description, project_id, project["owner_id"]),
    )
    return {"data": await owned_project(request, project_id)}

@projects.delete("/<int:project_id>")
async def delete_project(request, project_id):
    await check_csrf(request)
    project = await owned_project(request, project_id)
    await request.app.course_db.execute(
        "DELETE FROM projects WHERE id = ? AND owner_id = ?",
        (project_id, project["owner_id"]),
    )
    return {"data": {"deleted": True}}

```

modules/tasks/module.py

File: modules/tasks/module.py

```

"""Tasks belong to projects; project ownership guards every task operation."""

from flaxon.modules import FlaxonModule
from flaxon.http import JSONResponse
from flaxon.exceptions import NotFound, BadRequest
from security import check_csrf
from validation import json_object, task_fields
from modules.projects.module import owned_project

tasks = FlaxonModule("tasks")

async def owned_task(request, task_id):
    task = await request.app.course_db.one(
        "SELECT * FROM tasks WHERE id = ?", (task_id,)
    )
    if task is None:
        raise NotFound("Task not found.")
    await owned_project(request, task["project_id"])
    return task

@tasks.get("/project/<int:project_id>")
async def list_tasks(request, project_id):
    await owned_project(request, project_id)
    status = request.query.get("status")
    if status and status not in ("todo", "doing", "done"):
        raise BadRequest("Unknown task status.")
    sql = "SELECT * FROM tasks WHERE project_id = ?"
    parameters = [project_id]
    if status:
        sql += " AND status = ?"
        parameters.append(status)
    return {

        "data": await request.app.course_db.all(sql + " ORDER BY id DESC", parameters)
    }

```

```

    }

@tasks.post("/project/<int:project_id>")
async def create_task(request, project_id):
    await check_csrf(request)
    await owned_project(request, project_id)
    data = task_fields(await json_object(request))
    task_id = await request.app.course_db.execute(
        "INSERT INTO tasks(project_id, title, status, due_date) VALUES (?, ?, ?, ?)",
        (project_id, data["title"], data["status"], data["due_date"]),
    )
    return JSONResponse({"data": await owned_task(request, task_id)}, status_code=201)

@tasks.patch("/<int:task_id>")
async def update_task(request, task_id):
    await check_csrf(request)
    current = await owned_task(request, task_id)
    data = task_fields(await json_object(request), partial=True)
    updated = {**current, **data}
    await request.app.course_db.execute(
        "UPDATE tasks SET title = ?, status = ?, due_date = ? WHERE id = ?",
        (updated["title"], updated["status"], updated["due_date"], task_id),
    )
    return {"data": await owned_task(request, task_id)}

@tasks.delete("/<int:task_id>")
async def delete_task(request, task_id):
    await check_csrf(request)
    await owned_task(request, task_id)
    await request.app.course_db.execute("DELETE FROM tasks WHERE id = ?", (task_id,))

    return {"data": {"deleted": True}}

```

backoffice.py

File: backoffice.py

```

"""Staff model adapters reuse the domain database; CMS owns only editorial content."""

from flaxon.admin import AdminDashboard, AdminConfig
from flaxon.admin.registry import Registry
from flaxon.admin.cms import CMS, ContentType, CMSField
from flaxon.exceptions import BadRequest
from validation import text, task_fields

def configure_backoffice(app, admin_path, uploads_path):
    admin = AdminDashboard(
        app,
        registry=Registry(),
        users=[],
        storage_path=str(admin_path),
        upload_dir=str(uploads_path),
        strict_permissions=True,
        microservices=False,
        config=AdminConfig(site_title="Project Manager Operations"),
    )

    class ProjectAdmin:
        @classmethod
        async def get_instances(cls):

```

```

        return await app.course_db.all("SELECT * FROM projects ORDER BY id DESC")

    @classmethod
    async def get_instance(cls, object_id):
        return await app.course_db.one(
            "SELECT * FROM projects WHERE id = ?", (object_id,)
        )

    @classmethod
    async def create_instance(cls, data):

        raise BadRequest(
            "Create projects through the application to assign an owner."
        )

    @classmethod
    async def update_instance(cls, object_id, data):
        current = await cls.get_instance(object_id)
        if current is None:
            return None
        name = text(data, "name")
        description = text(data, "description", 1000, required=False)
        await app.course_db.execute(
            "UPDATE projects SET name = ?, description = ? WHERE id = ?",
            (name, description, object_id),
        )
        return await cls.get_instance(object_id)

    @classmethod
    async def delete_instance(cls, object_id):
        current = await cls.get_instance(object_id)
        if not current:
            return False
        await app.course_db.execute(
            "DELETE FROM projects WHERE id = ?", (object_id,)
        )
        return True

class TaskAdmin:
    @classmethod
    async def get_instances(cls):
        return await app.course_db.all("SELECT * FROM tasks ORDER BY id DESC")

    @classmethod
    async def get_instance(cls, object_id):

        return await app.course_db.one(
            "SELECT * FROM tasks WHERE id = ?", (object_id,)
        )

    @classmethod
    async def create_instance(cls, data):
        raise BadRequest("Create tasks through their project in the application.")

    @classmethod
    async def update_instance(cls, object_id, data):
        current = await cls.get_instance(object_id)
        if current is None:
            return None
        values = {**current, **task_fields(data, partial=True)}
        await app.course_db.execute(
            "UPDATE tasks SET title = ?, status = ?, due_date = ? WHERE id = ?",
            (values["title"], values["status"], values["due_date"], object_id),
        )

```

```

        return await cls.get_instance(object_id)

    @classmethod
    async def delete_instance(cls, object_id):
        current = await cls.get_instance(object_id)
        if not current:
            return False
        await app.course_db.execute("DELETE FROM tasks WHERE id = ?", (object_id,))
        return True

admin.register(
    ProjectAdmin,
    name="project",
    list_display=["id", "name", "owner_id"],
    search_fields=["name"],
    fields=["name", "description"],

    readonly_fields=["id", "owner_id"],
)
admin.register(
    TaskAdmin,
    name="task",
    list_display=["id", "title", "status", "due_date"],
    search_fields=["title"],
    list_filter=["status"],
    fields=["title", "status", "due_date"],
    readonly_fields=["id", "project_id"],
)
cms = CMS(app, auth=admin.auth)
cms.register(
    ContentType(
        "help_article",
        label="Help article",
        label_plural="Help articles",
        fields=[
            CMSField("title", required=True),
            CMSField("summary", type="textarea"),
            CMSField("body", type="richtext"),
        ],
        statuses=["draft", "published"],
        list_display=["title", "status"],
    )
)
cms.register(
    ContentType(
        "announcement",
        fields=[
            CMSField("title", required=True),
            CMSField("body", type="textarea"),
        ],
        statuses=["draft", "published"],
    )
)
app.backoffice, app.cms = admin, cms

```

modules/content/module.py

File: modules/content/module.py

```

"""Public read-only content endpoints; never expose the authenticated CMS API to readers."""

```

```

from pathlib import Path
from flaxon.modules import FlaxonModule

```

```

from flaxon.exceptions import NotFound

content = FlaxonModule(
    "content",
    ui_dir=Path(__file__).parent / "ui",
    ui_routes={"Help.js": "/help", "Article/[slug].js": "/help/:slug"},
)

async def published_articles(request):
    cms = request.app.cms
    await cms._load_database()
    articles = cms.content_types["help_article"]
    return [
        item for item in articles.items.values() if item.get("status") == "published"
    ]

@content.get("/articles")
async def list_articles(request):
    articles = await published_articles(request)
    return {
        "data": [
            {key: item.get(key) for key in ("id", "title", "slug", "summary")}
            for item in articles
        ]
    }

@content.get("/articles/<slug>")
async def get_article(request, slug):
    for article in await published_articles(request):
        if article["slug"] == slug:
            return {
                "data": {
                    key: article.get(key)
                    for key in ("id", "title", "slug", "summary", "body")
                }
            }
    raise NotFound("Article not found.")

```

seed.py

File: seed.py

```

"""Idempotent sample content. Register your own user before running this command."""

from app import create_app

async def seed():
    app = create_app()
    user = await app.course_db.one("SELECT id FROM users ORDER BY id LIMIT 1")
    if not user:
        raise ValueError(
            "Register an account in the browser before seeding sample projects."
        )
    if not await app.course_db.one(
        "SELECT id FROM projects WHERE owner_id = ?", (user["id"],)
    ):
        project_id = await app.course_db.execute(
            "INSERT INTO projects(owner_id, name, description) VALUES (?, ?, ?)",
            (

```

```

        user["id"],
        "Launch my portfolio",
        "A small project to practise planning.",
    ),
)
for title, status in [
    ("Choose a design", "done"),
    ("Build the homepage", "doing"),
    ("Deploy the website", "todo"),
]:
    await app.course_db.execute(
        "INSERT INTO tasks(project_id, title, status) VALUES (?, ?, ?)",
        (project_id, title, status),
    )
articles = app.cms.content_types["help_article"]
if not articles.items:

    articles.create(
        {
            "title": "Getting started",
            "summary": "Create a project, add tasks, and track progress.",
            "body": "<p>Open Projects, create a project, and add your first task. Move
tasks from To do to Doing to Done.</p>",
            "status": "published",
        }
    )
    app.cms._save(articles)
print("Sample projects, tasks, and help content are ready.")

```

ui/types.ts

File: ui/types.ts

```

export interface User { id: number; name: string; email: string; }
export interface Project { id: number; owner_id: number; name: string; description: string; }
export type TaskStatus = 'todo' | 'doing' | 'done';
export interface Task { id: number; project_id: number; title: string; status: TaskStatus;
    due_date: string | null; }
export interface Session { user: User | null; csrf: string; }
export interface Article { id: string; title: string; slug: string; summary: string; body?:
    string; }

```

ui/api.ts

File: ui/api.ts

```

import type { Session } from './types';

let session: Session | null = null;
export async function loadSession(): Promise<Session> {
    const response = await fetch('/api/auth/session', { credentials: 'same-origin' });
    const payload = await response.json();
    if (!response.ok) throw new Error(payload.error?.message || 'Could not load your
session. ');
    session = payload.data;
    return session!;
}

export async function api<T>(path: string, method = 'GET', body?: unknown): Promise<T> {
    if (!session) await loadSession();
    const response = await fetch(path, {
        method,
        credentials: 'same-origin',
        headers: { 'Content-Type': 'application/json', 'X-CSRF-Token': session!.csrf },
    });

```

```

    body: body === undefined ? undefined : JSON.stringify(body),
  });
  const payload = await response.json();
  if (!response.ok) {
    if (response.status === 401 || response.status === 403) session = null;
    throw new Error(payload.error?.message || 'The request failed. Please try again.');
```

ui/app.html

File: ui/app.html

```

<template>
  <div class="app-shell">
    <header><a href="/" data-teloc-link class="brand">Project Manager</a>
    <nav aria-label="Main navigation">
      <a href="/projects" data-teloc-link>Projects</a>
      <a href="/help" data-teloc-link>Help</a>
      <a href="/login" data-teloc-link>Account</a>
    </nav>
  </header>
  <main id="router-view" data-teloc-router-view></main>
  <footer>Built with Flaxon and Teloce. <a href="/admin/">Staff Admin</a></footer>
</div>
</template>
<script lang="ts">export default {};</script>
<style scoped>
/* Only the document reset is global: body is outside this component. */
:global(body) {
  margin: 0;
}
.app-shell {
  min-height: 100vh;
  font-family: system-ui, sans-serif;
  color: #17243a;
  background: #f3f6fc;
}
header {
  display: flex;
  justify-content: space-between;
  gap: 1rem;
  align-items: center;
  padding: 1.2rem max(1rem, calc((100vw - 1080px) / 2));
  background: #17243a;
  color: white;
}
header a {
  color: white;
  text-decoration: none;
}
.brand {
  font-weight: 800;
  font-size: 1.3rem;
}
nav {
  display: flex;
  gap: 1rem;
  flex-wrap: wrap;
}
main {
```

```

    max-width: 1080px;
    margin: auto;
    min-height: 75vh;
    padding: 2rem 1rem;
}
footer {
    padding: 1.5rem;
    text-align: center;
    color: #526079;
}
footer a {
    color: #1d4ed8;
}
:focus-visible {
    outline: 3px solid #f59e0b;
    outline-offset: 3px;
}
@media (max-width: 520px) {
    header {
        flex-direction: column;

        align-items: flex-start;
    }
    main {
        padding: 1rem;
    }
}
</style>

```

ui/pages/Home.html

File: ui/pages/Home.html

```

<template><section class="hero">
  <p class="eyebrow">A practical Python full-stack course</p>
  <h1>Make room for your next idea.</h1>
  <p>Create projects, break them into tasks, and watch your progress.</p>
  <a class="button" href="/projects" data-teloc-link>Open projects</a>
  <a href="/login" data-teloc-link>Create an account</a>
</section></template>
<script lang="ts">export default {};</script>
<style scoped>
* {
  box-sizing: border-box;
}
a {
  color: #1d4ed8;
}
h1 {
  font-size: clamp(1.8rem, 5vw, 3rem);
  line-height: 1.15;
}
h2 {
  font-size: 1.2rem;
}
p {
  line-height: 1.6;
}
[role="alert"] {
  color: #a21c2b;
  padding: 0.8rem;
  border-left: 4px solid;
}
:focus-visible {

```

```

outline: 3px solid #f59e0b;
outline-offset: 3px;
}

.hero {
  max-width: 700px;
  padding: 3rem 0;
}
.eyebrow {
  text-transform: uppercase;
  font-size: 0.8rem;
  letter-spacing: 0.1em;
}
.button {
  display: inline-block;
  border-radius: 7px;
  min-height: 44px;
  background: #2563eb;
  color: white;
  padding: 0.7rem 1rem;
  text-decoration: none;
  margin-right: 0.5rem;
}
</style>

```

modules/auth/ui/pages/Login.html

File: modules/auth/ui/pages/Login.html

```

<template>
  <section class="narrow"><h1>Your account</h1>
    <p v-if="busy" role="status">Please wait...</p>
    <p v-if="message" role="alert">{{ message }}</p>
    <div v-if="user"><p>Signed in as {{ user.name }}</p>
      <a href="/projects" data-teloc-link>Open projects</a>
      <button type="button" @click="logout" :disabled="busy">Sign out</button>
    </div>
    <form v-show="!user" @submit.prevent="submit">
      <label v-show="registering">Name<input name="name" v-model="name" maxlength="80"
      :required="registering" autocomplete="name" /></label>
      <label>Email<input name="email" type="email" v-model="email" required
      autocomplete="username" /></label>
      <label>Password<input name="password" type="password" v-model="password" minlength="12"
      maxlength="128" required autocomplete="current-password" /></label>
      <button :disabled="busy">{{ registering ? 'Create account' : 'Sign in' }}</button>
      <button type="button" class="secondary" @click="registering = !registering"
      :disabled="busy">{{ registering ? 'Use an existing account' : 'Register instead'
      }}</button>
    </form>
  </section>
</template>
<script lang="ts">
import { api, loadSession } from '../../../../ui/api.ts';
import type { Session } from '../../../../ui/types.ts';
export default {
  data() { return { user: null, registering: false, name: '', email: '', password: '', busy:
    false, message: '' }; },
  async mounted() {
    try { this.user = (await loadSession()).user; }
    catch (error) { this.message = error.message; }
  },
  methods: {
    async submit() {

```

```

    if (this.busy) return;
    this.busy = true; this.message = '';
    try {
      const path = this.registering ? '/api/auth/register' : '/api/auth/login';
      const result = await api<Session>(path, 'POST', { name: this.name, email: this.email,
password: this.password });
      this.user = result.user;
      this.password = '';
      window.__TELOCE_ROUTER__.navigate('/projects');
    } catch (error) { this.message = error.message; }
    finally { this.busy = false; }
  },
  async logout() {
    this.busy = true;
    try { await api('/api/auth/logout', 'POST', {}); this.user = null; this.message =
'Signed out.'; }
    catch (error) { this.message = error.message; }
    finally { this.busy = false; }
  },
},
};
</script>
<style scoped>
* {
  box-sizing: border-box;
}
a {
  color: #1d4ed8;
}
h1 {
  font-size: clamp(1.8rem, 5vw, 3rem);
  line-height: 1.15;
}
h2 {

  font-size: 1.2rem;
}
p {
  line-height: 1.6;
}
[role="alert"] {
  color: #a21c2b;
  padding: 0.8rem;
  border-left: 4px solid;
}
:focus-visible {
  outline: 3px solid #f59e0b;
  outline-offset: 3px;
}
label {
  display: grid;
  gap: 0.4rem;
  margin: 0.9rem 0;
  font-weight: 600;
}
input, select, textarea, button {
  font: inherit;
  border-radius: 7px;
  padding: 0.7rem;
  min-height: 44px;
}
input, select, textarea {
  width: 100%;
  border: 1px solid #abb9cf;
  background: white;

```

```

    color: #17243a;
}
button {
  display: inline-block;

  border: 0;
  background: #2563eb;
  color: white;
  padding: 0.7rem 1rem;
  cursor: pointer;
  margin: 0.3rem 0.5rem 0.3rem 0;
}
button:disabled {
  opacity: 0.55;
  cursor: wait;
}
.secondary {
  background: #e4eaf5;
  color: #17243a;
}
.danger {
  background: #b42332;
}
.narrow {
  max-width: 480px;
  margin: auto;
}
</style>

```

modules/projects/ui/pages/ProjectList.html

File: modules/projects/ui/pages/ProjectList.html

```

<template><section>
  <h1>Your projects</h1><p>Choose a project to manage its tasks.</p>
  <p v-if="message" role="alert">{{ message }} <a href="/login"
    data-teloc-link>Account</a></p>
  <p v-if="loading" role="status">Loading projects...</p>
  <form @submit.prevent="createProject" class="card">
    <h2>Create a project</h2>
    <label>Name<input name="project-name" v-model="name" required maxlength="120" /></label>
    <label>Description<textarea v-model="description" maxlength="1000"></textarea></label>
    <button :disabled="busy">{{ busy ? 'Saving...' : 'Create project' }}</button>
  </form>
  <p v-if="!loading && projects.length === 0">No projects yet. Create your first one
    above.</p>
  <div class="cards"><a v-for="project in projects" :key="project.id" :href="'/projects/' +
    project.id" data-teloc-link class="card">
    <h2>{{ project.name }}</h2><p>{{ project.description }}</p>
  </a></div>
</section></template>
<script lang="ts">
import { api } from '../../../ui/api.ts';
import type { Project } from '../../../ui/types.ts';
export default {
  data() { return { projects: [], name: '', description: '', message: '', loading: true, busy:
    false }; },
  async mounted() {
    try { this.projects = await api<Project[]>('/api/projects/'); }
    catch (error) { this.message = error.message; }
    finally { this.loading = false; }
  },
  methods: {
    async createProject() {

```

```

    if (this.busy) return;
    this.busy = true; this.message = '';
    try {

        const project = await api<Project>('/api/projects/', 'POST', { name: this.name,
        description: this.description });
        this.projects = [project, ...this.projects]; this.name = ''; this.description = '';
    } catch (error) { this.message = error.message; }
    finally { this.busy = false; }
  },
},
};
</script>
<style scoped>
* {
  box-sizing: border-box;
}
a {
  color: #1d4ed8;
}
h1 {
  font-size: clamp(1.8rem, 5vw, 3rem);
  line-height: 1.15;
}
h2 {
  font-size: 1.2rem;
}
p {
  line-height: 1.6;
}
[role="alert"] {
  color: #a21c2b;
  padding: 0.8rem;
  border-left: 4px solid;
}
:focus-visible {
  outline: 3px solid #f59e0b;
  outline-offset: 3px;
}
.card {
  display: block;
  background: white;
  border: 1px solid #d8e0ed;
  border-radius: 12px;
  padding: 1.2rem;
  margin: 1rem 0;
  color: inherit;
  text-decoration: none;
}
label {
  display: grid;
  gap: 0.4rem;
  margin: 0.9rem 0;
  font-weight: 600;
}
input, select, textarea, button {
  font: inherit;
  border-radius: 7px;
  padding: 0.7rem;
  min-height: 44px;
}
input, select, textarea {
  width: 100%;

```

```

border: 1px solid #abb9cf;
background: white;
color: #17243a;
}
button {
  display: inline-block;
  border: 0;
  background: #2563eb;
  color: white;

  padding: 0.7rem 1rem;
  cursor: pointer;
  margin: 0.3rem 0.5rem 0.3rem 0;
}
button:disabled {
  opacity: 0.55;
  cursor: wait;
}
.secondary {
  background: #e4eaf5;
  color: #17243a;
}
.danger {
  background: #b42332;
}
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 240px), 1fr));
  gap: 1rem;
}
</style>

```

modules/projects/ui/pages/ProjectDetails/[id].html

File: modules/projects/ui/pages/ProjectDetails/[id].html

```

<template><section>
  <a href="/projects" data-teloc-link><- All projects</a>
  <p v-if="loading" role="status">Loading project...</p><p v-if="message" role="alert">{{
    message }}</p>
  <div v-if="project">
    <h1>{{ project.name }}</h1><p>{{ project.description }}</p>
    <p aria-live="polite">{{ progress }}% complete</p><progress :value="progress" max="100"
      aria-label="Project progress"></progress>
    <form @submit.prevent="saveProject" class="card">
      <h2>Edit project</h2>
      <label>Name<input v-model="name" required maxlength="120" /></label>
      <label>Description<textarea v-model="description" maxlength="1000"></textarea></label>
      <button :disabled="busy">Save project</button>
      <button type="button" class="danger" @click="deleteProject" :disabled="busy">Delete
        project</button>
    </form>
    <TaskForm :busy="busy" @create="createTask" />
    <label>Filter tasks<select v-model="filter"><option value="all">All tasks</option><option
      value="todo">To do</option><option value="doing">Doing</option><option
      value="done">Done</option></select></label>
    <TaskList :tasks="visibleTasks" :busy="busy" @status="changeStatus" @remove="removeTask"
      />
  </div>
</section></template>
<script lang="ts">
import TaskForm from '../../components/TaskForm.html';
import TaskList from '../../components/TaskList.html';
import { api } from '../../../../ui/api.ts';

```

```

import type { Project, Task } from '../../../../ui/types.ts';

// The compiler imports these signal helpers automatically for this .html component.
const taskState = signal([]);
const completion = computed(() => {
  const tasks = taskState();

  return tasks.length ? Math.round(tasks.filter(task => task.status === 'done').length * 100 /
    tasks.length) : 0;
});
export default {
  props: ['id'], components: { TaskForm, TaskList },
  data() { return { project: null, tasks: [], name: '', description: '', filter: 'all',
    progress: 0, busy: false, loading: true, message: '' }; },
  computed: { visibleTasks() { return this.filter === 'all' ? this.tasks :
    this.tasks.filter(task => task.status === this.filter); } },
  async mounted() {
    taskState.set([]);
    this.progressEffect = effect(() => { this.progress = completion(); });
    try {
      const [project, tasks] = await Promise.all([api<Project>('/api/projects/' + this.id),
        api<Task[]>('/api/tasks/project/' + this.id)]);
      this.project = project; this.name = project.name; this.description =
        project.description;
      this.setTasks(tasks);
    } catch (error) { this.message = error.message; }
    finally { this.loading = false; }
  },
  beforeUnmount() { this.progressEffect?.stop(); },
  methods: {
    setTasks(tasks) { this.tasks = tasks; taskState.set(tasks); },
    async mutate(operation) {
      if (this.busy) return;
      this.busy = true; this.message = '';
      try { await operation(); }
      catch (error) { this.message = error.message; }
      finally { this.busy = false; }
    },
    async saveProject() {
      await this.mutate(async () => { this.project = await api<Project>('/api/projects/' +
        this.id, 'PUT', { name: this.name, description: this.description }); });
    },
    async deleteProject() {
      if (!window.confirm('Delete this project and all its tasks?')) return;
      await this.mutate(async () => { await api('/api/projects/' + this.id, 'DELETE', {});
        window.__TELOCE_ROUTER__.navigate('/projects'); });
    },
    async createTask(data) {
      await this.mutate(async () => { const task = await api<Task>('/api/tasks/project/' +
        this.id, 'POST', data); this.setTasks([task, ...this.tasks]); });
    },
    async changeStatus(data) {
      await this.mutate(async () => { const updated = await api<Task>('/api/tasks/' + data.id,
        'PATCH', { status: data.status }); this.setTasks(this.tasks.map(task => task.id ===
        updated.id ? updated : task)); });
    },
    async removeTask(id) {
      if (!window.confirm('Delete this task?')) return;
      await this.mutate(async () => { await api('/api/tasks/' + id, 'DELETE', {});
        this.setTasks(this.tasks.filter(task => task.id !== id)); });
    },
  },
};

```

```

</script>
<style scoped>
* {
  box-sizing: border-box;
}
a {
  color: #1d4ed8;
}
h1 {
  font-size: clamp(1.8rem, 5vw, 3rem);
  line-height: 1.15;
}

h2 {
  font-size: 1.2rem;
}
p {
  line-height: 1.6;
}
[role="alert"] {
  color: #a21c2b;
  padding: 0.8rem;
  border-left: 4px solid;
}
:focus-visible {
  outline: 3px solid #f59e0b;
  outline-offset: 3px;
}
.card {
  display: block;
  background: white;
  border: 1px solid #d8e0ed;
  border-radius: 12px;
  padding: 1.2rem;
  margin: 1rem 0;
  color: inherit;
  text-decoration: none;
}
label {
  display: grid;
  gap: 0.4rem;
  margin: 0.9rem 0;
  font-weight: 600;
}
input, select, textarea, button {
  font: inherit;
  border-radius: 7px;

  padding: 0.7rem;
  min-height: 44px;
}
input, select, textarea {
  width: 100%;
  border: 1px solid #abb9cf;
  background: white;
  color: #17243a;
}
button {
  display: inline-block;
  border: 0;
  background: #2563eb;
  color: white;
  padding: 0.7rem 1rem;
  cursor: pointer;
}

```

```

    margin: 0.3rem 0.5rem 0.3rem 0;
  }
  button:disabled {
    opacity: 0.55;
    cursor: wait;
  }
  .secondary {
    background: #e4eaf5;
    color: #17243a;
  }
  .danger {
    background: #b42332;
  }
  progress {
    width: 100%;
    height: 1.2rem;
    accent-color: #2563eb;
  }
</style>

```

modules/projects/ui/components/TaskForm.html

File: modules/projects/ui/components/TaskForm.html

```

<template><form @submit.prevent="submit" class="card">
  <h2>Add a task</h2>
  <label>Task title<input name="task-title" v-model="title" maxlength="160" required
    /></label>
  <label>Due date<input name="due-date" type="date" v-model="dueDate" /></label>
  <button :disabled="busy">Add task</button>
</form></template>
<script lang="ts">
export default {
  props: ['busy'], emits: ['create'],
  data() { return { title: '', dueDate: '' }; },
  methods: {
    submit() {
      if (this.busy || !this.title.trim()) return;
      this.$emit('create', { title: this.title, due_date: this.dueDate || null });
    },
  },
};
</script>
<style scoped>
* {
  box-sizing: border-box;
}
a {
  color: #1d4ed8;
}
h1 {
  font-size: clamp(1.8rem, 5vw, 3rem);
  line-height: 1.15;
}
h2 {
  font-size: 1.2rem;
}
p {
  line-height: 1.6;
}
[role="alert"] {
  color: #a21c2b;
}

```

```

padding: 0.8rem;
border-left: 4px solid;
}
:focus-visible {
  outline: 3px solid #f59e0b;
  outline-offset: 3px;
}
.card {
  display: block;
  background: white;
  border: 1px solid #d8e0ed;
  border-radius: 12px;
  padding: 1.2rem;
  margin: 1rem 0;
  color: inherit;
  text-decoration: none;
}
label {
  display: grid;
  gap: 0.4rem;
  margin: 0.9rem 0;
  font-weight: 600;
}
input, select, textarea, button {
  font: inherit;
  border-radius: 7px;
  padding: 0.7rem;
  min-height: 44px;
}
input, select, textarea {
  width: 100%;
  border: 1px solid #abb9cf;
  background: white;
  color: #17243a;
}
button {
  display: inline-block;
  border: 0;
  background: #2563eb;
  color: white;
  padding: 0.7rem 1rem;
  cursor: pointer;
  margin: 0.3rem 0.5rem 0.3rem 0;
}
button:disabled {
  opacity: 0.55;
  cursor: wait;
}
.secondary {
  background: #e4eaf5;
  color: #17243a;
}
.danger {
  background: #b42332;
}
</style>

```

modules/projects/ui/components/TaskList.html

File: modules/projects/ui/components/TaskList.html

```

<template><div>
  <p v-if="tasks.length === 0">No tasks match this filter.</p>

```

```

<ul class="task-list"><li v-for="task in tasks" :key="task.id" class="card">
  <strong>{{ task.title }}</strong><span>{{ task.due_date ? 'Due ' + task.due_date : 'No due
    date' }}</span>
  <label>Status<select aria-label="Task status" :value="task.status"
    @change="$emit('status', { id: task.id, status: $event.target.value })" :disabled="busy">
    <option value="todo">To do</option><option value="doing">Doing</option><option
    value="done">Done</option>
  </select></label>
  <button class="danger" type="button" @click="$emit('remove', task.id)"
    :disabled="busy">Delete task</button>
</li></ul>
</div></template>
<script lang="ts">export default { props: ['tasks', 'busy'], emits: ['status', 'remove']
  };</script>
<style scoped>
* {
  box-sizing: border-box;
}
a {
  color: #1d4ed8;
}
h1 {
  font-size: clamp(1.8rem, 5vw, 3rem);
  line-height: 1.15;
}
h2 {
  font-size: 1.2rem;
}
p {
  line-height: 1.6;
}
[role="alert"] {
  color: #a21c2b;
  padding: 0.8rem;
  border-left: 4px solid;
}
:focus-visible {
  outline: 3px solid #f59e0b;
  outline-offset: 3px;
}
.card {
  display: block;
  background: white;
  border: 1px solid #d8e0ed;
  border-radius: 12px;
  padding: 1.2rem;
  margin: 1rem 0;
  color: inherit;
  text-decoration: none;
}
label {
  display: grid;
  gap: 0.4rem;
  margin: 0.9rem 0;
  font-weight: 600;
}
input, select, textarea, button {
  font: inherit;
  border-radius: 7px;
  padding: 0.7rem;
  min-height: 44px;
}
input, select, textarea {
  width: 100%;

```

```

border: 1px solid #abb9cf;
background: white;

color: #17243a;
}
button {
display: inline-block;
border: 0;
background: #2563eb;
color: white;
padding: 0.7rem 1rem;
cursor: pointer;
margin: 0.3rem 0.5rem 0.3rem 0;
}
button:disabled {
opacity: 0.55;
cursor: wait;
}
.secondary {
background: #e4eaf5;
color: #17243a;
}
.danger {
background: #b42332;
}
.task-list {
list-style: none;
padding: 0;
}
.task-list span {
display: block;
color: #526079;
margin-top: 0.5rem;
}
</style>

```

modules/content/ui/pages/Help.html

File: modules/content/ui/pages/Help.html

```

<template><section><h1>Help centre</h1>
  <p v-if="loading" role="status">Loading articles...</p><p v-if="message" role="alert">{{
    message }}</p>
  <p v-if="!loading && articles.length === 0">No published articles yet.</p>
  <div class="cards"><a v-for="article in articles" :key="article.id" :href="'/help/' +
    article.slug" data-teloce-link class="card"><h2>{{ article.title }}</h2><p>{{
    article.summary }}</p></a></div>
</section></template>
<script lang="ts">
import { api } from '../../../ui/api.ts';
export default {
  data() { return { articles: [], loading: true, message: '' }; },
  async mounted() {
    try { this.articles = await api('/api/content/articles'); }
    catch (error) { this.message = error.message; }
    finally { this.loading = false; }
  },
};
</script>
<style scoped>
* {
  box-sizing: border-box;
}
a {

```

```

    color: #1d4ed8;
  }
  h1 {
    font-size: clamp(1.8rem, 5vw, 3rem);
    line-height: 1.15;
  }
  h2 {
    font-size: 1.2rem;
  }
  p {
    line-height: 1.6;
  }
  [role="alert"] {
    color: #a21c2b;
    padding: 0.8rem;
    border-left: 4px solid;
  }
  :focus-visible {
    outline: 3px solid #f59e0b;
    outline-offset: 3px;
  }
  .card {
    display: block;
    background: white;
    border: 1px solid #d8e0ed;
    border-radius: 12px;
    padding: 1.2rem;
    margin: 1rem 0;
    color: inherit;
    text-decoration: none;
  }
  .cards {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(min(100%, 240px), 1fr));
    gap: 1rem;
  }
</style>

```

modules/content/ui/pages/Article/[slug].html

File: modules/content/ui/pages/Article/[slug].html

```

<template><article><a href="/help" data-teloc-link><- Help centre</a>
  <p v-if="loading" role="status">Loading article...</p><p v-if="message" role="alert">{{
    message }}</p>
  <div v-if="article"><h1>{{ article.title }}</h1><p>{{ article.summary }}</p><div
    class="article-body" v-html="article.body"></div></div>
</article></template>
<script lang="ts">
import { api } from '../../../../../ui/api.ts';
export default {
  props: ['slug'], data() { return { article: null, loading: true, message: '' }; },
  async mounted() {
    try { this.article = await api('/api/content/articles/' + encodeURIComponent(this.slug));
    }
    catch (error) { this.message = error.message; }
    finally { this.loading = false; }
  },
};
</script>
<style scoped>
* {
  box-sizing: border-box;

```

```

}
a {
  color: #1d4ed8;
}
h1 {
  font-size: clamp(1.8rem, 5vw, 3rem);
  line-height: 1.15;
}
h2 {
  font-size: 1.2rem;
}
p {
  line-height: 1.6;
}

[role="alert"] {
  color: #a21c2b;
  padding: 0.8rem;
  border-left: 4px solid;
}
:focus-visible {
  outline: 3px solid #f59e0b;
  outline-offset: 3px;
}
.article-body {
  line-height: 1.6;
  overflow-wrap: anywhere;
}
</style>

```

.env.example

File: .env.example

```

FLAXON_DEBUG=1
PUBLIC_ORIGIN=http://127.0.0.1:8000
# Production: FLAXON_DEBUG=0, PUBLIC_ORIGIN=https://your-app.onrender.com
# Render persistent disk mount: DATA_DIR=/var/data
# Generate with: python -c "import secrets; print(secrets.token_urlsafe(48))"
FLAXON_SECRET_KEY=

```

requirements.txt

File: requirements.txt

```

-c requirements.lock.txt
./vendor/flaxon-0.2.6+course.1-py3-none-any.whl
./vendor/telece_py-0.2.6-py3-none-any.whl
minifyjs==0.1.3
uvicorn==0.38.0
Jinja2==3.1.6
aiosqlite==0.21.0
argon2-cffi==25.1.0
python-dotenv==1.2.1
nh3==0.3.7

```

requirements-dev.txt

File: requirements-dev.txt

```

-r requirements.txt
pytest==9.1.1
httpx==0.28.1
playwright==1.51.0

```

scripts/build_ui.py

File: scripts/build_ui.py

```
"""Build production Teloce assets without touching the live database or Render disk."""

import os
import secrets
import sys
import tempfile
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
with tempfile.TemporaryDirectory() as build_data:
    os.environ["DATA_DIR"] = build_data
    os.environ["FLAXON_DEBUG"] = "0"
    os.environ.setdefault("FLAXON_SECRET_KEY", secrets.token_urlsafe(48))
    from app import app

    result = app.teloc.build()
    print(f"Built {result.get('compiled', 0)} Teloce components with MinifyJS.")
```

render.yaml

File: render.yaml

```
services:
  - type: web
    name: flaxon-project-manager
    runtime: python
    plan: starter
    buildCommand: python scripts/verify_vendor.py && pip install -r requirements.txt && python
      scripts/build_ui.py
    startCommand: python management.py migrate && python -m uvicorn app:app --host 0.0.0.0
    --port $PORT --workers 1
    healthCheckPath: /health/course
    disk:
      name: project-manager-data
      mountPath: /var/data
      sizeGB: 1
    envVars:
      - key: FLAXON_DEBUG
        value: "0"
      - key: FLAXON_SECRET_KEY
        generateValue: true
      - key: DATA_DIR
        value: /var/data
      - key: PUBLIC_ORIGIN
        sync: false
```

Appendix B | Recording your sample lesson

Sample lesson: one project API to a Teloce screen

Target: 12-15 minutes. Record with your own voice. This file is a rehearsal script, not a completed recording. The finished app already contains the feature; use a throwaway teaching checkout when removing code to type it again.

Before recording

Install using the README, migrate, and run the development server. Register a browser account. Close personal tabs, hide credentials, set the editor to a readable font, and test your microphone. Capture 1080p if your equipment supports it; prioritize legible code and clean audio. Keep the existing scoped component blocks supplied. Teach a few local rules as part of the interface take; there is no shared app.css file. Run `python scripts/course_api_demo.py` and the ownership test before the take. The demo deliberately creates a new sample account and project on each run.

0:00-1:00 | Show the result

Say: "We will create a project through a protected Flaxon API, test its ownership rule, then display it in a Teloce HTML screen. The browser cannot choose who owns this project. The server gets that from the signed-in session."

Show `/projects`, create a project, and open it. Explain the JSON data envelope.

1:00-4:00 | Type the route

Open `modules/projects/module.py`. Type `create_project` while explaining each step: check CSRF, require a user, parse a JSON object, validate name/description, insert with SQL placeholders, return the owned record with 201. Show `owned_project` and point to both the ID and `owner_id` conditions. Do not skip imports or pretend the session/database helper has not been supplied by earlier chapters.

Say: "A name in a request is a value. A question mark lets SQLite treat it as a value instead of SQL. The session decides `owner_id`."

4:00-6:00 | Exercise HTTP

In a second terminal run `python scripts/course_api_demo.py`. Show the created record and the 400 for a blank name. Explain that registration rotates CSRF; the demo reads the new token before creating the project. Its account is separate from your browser account.

6:00-8:00 | Test ownership

Open `tests/test_api.py` and explain `test_project_crud_and_ownership`: create one project, register a different user, and expect 404 for read/update/delete. Run:

```
python -m pytest -q tests/test_api.py -k project_crud_and_ownership
```

Say: "A hidden button cannot protect data. This test calls the API directly as another user. The server must deny it."

8:00-12:00 | Connect the screen

Open `modules/projects/ui/pages/ProjectList.html`. Explain the api import, mounted GET request, projects array, loading/error messages, and v-for cards. Type the `createProject` method, form submit binding, and a few rules in the page's style scoped block. The helper unwraps data and sends cookies plus CSRF. On success prepend the returned project and clear the fields. Show the busy guard and finally block so failure does not leave the button stuck.

Show `ProjectList.html`'s scoped `.card` styles and `TaskForm.html`'s own button styles. Explain that inheritance still works, while page selectors do not style child component internals automatically. Then explain `data-telece-link`: an anchor to `/projects/ID` mounts the module page inside the SPA shell. A direct refresh also needs the Python shell route. Create a project in your browser, then use Back and refresh its detail page.

12:00-15:00 | Recap and exercise

Ask the viewer to submit a whitespace-only name through the API and add a test. Recap the path: form -> api helper -> Flaxon route -> owned database row -> JSON -> reactive screen. Mention that task signals and Admin/CMS follow in later lessons.

Feedback before recording the full course

Share the sample with two or three learners. Ask where they first became confused, which command they could not reproduce, and whether the font/audio were clear. Record timestamps, then revise the script and chapter before re-recording. Approval for YouTube publication is a separate decision by freeCodeCamp.

For each full-course chapter: show the goal, type meaningful behavior, demonstrate a success and a failure, run the relevant check, and point to the checkpoint. Avoid recording installation waits. End deployment with persistence and denied request checks rather than only a service-success screen.